Практическая работа №5. Машинное обучение в анализе данных с использованием Spark MLlib

Дисциплина	Технологии и инструментарий анализа больших данных
Студент	Дицель Н.С.
Группа	ИКБО-11-23
Проверил	старший преподаватель Приходько Н.А.
Дата выполнения	13.10.2026

Содержание

0. Окружение и SparkSession
1. Чтение и контроль исходных данных (разд. 3 методички)
2. Классификация покупательского намерения — Online Shoppers (разд. 4)
3. Регрессия спроса — Seoul Bike (разд. 5)
4. Кластеризация интернет-сессий — Online Shoppers (разд. 6)
5. Сохранение и повторное применение моделей (разд. 7)
6. Самостоятельная работа — регрессия времени прослушивания подкастов (разд. 8)
7. Ответы на контрольные вопросы (разд. 9)
8. Общие выводы

Блокнот выполняется сверху вниз после перезапуска ядра (Restart & Run All). Все модели обучаются в Spark MLlib; pandas / NumPy / Matplotlib / scikit-learn используются только для небольших итоговых таблиц, кривых и графиков.

0. Окружение и SparkSession

В методичке сессия считается уже созданной, поэтому здесь она создаётся явно в режиме `local[*]`. На Windows для записи моделей (`PipelineModel.save`) Spark использует Hadoop FileSystem, поэтому подключаются `winutils.exe` и `hadoop.dll`.

```
import os, sys, platform, subprocess
if os.name == "nt" and os.path.isdir(r"C:\hadoop\bin"):
    os.environ["HADOOP_HOME"] = r"C:\hadoop"
    os.environ["PATH"] = r"C:\hadoop\bin;" + os.environ["PATH"]
os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

import pyspark
from pyspark.sql import SparkSession

spark = (SparkSession.builder
    .master("local[*]")
    .appName("Practice5_MLlib")
    .config("spark.driver.memory", "4g")
    .config("spark.sql.shuffle.partitions", "8")
    .config("spark.ui.showConsoleProgress", "false")
    .getOrCreate())

java = subprocess.run(["java", "-version"], capture_output=True, text=True).stderr.splitlines()[0]
print("ОС     :", platform.platform())
print("Python :", sys.version.split()[0])
print("Java   :", java)
print("PySpark:", pyspark.__version__)
print("CPU    :", os.cpu_count())
```

```
ОС     : Windows-11-10.0.26300-SP0
Python : 3.12.10
Java   : openjdk version "21.0.12.1" 2026-08-18 LTS
PySpark: 3.5.3
CPU    : 16
```

1. Чтение и контроль исходных данных

Исходные файлы `online_shoppers_intention.csv` и `SeoulBikeData.csv` скачаны с UCI Machine Learning Repository (DOI 10.24432/C5F88Q и 10.24432/C5F62R) и лежат рядом с блокнотом.

Практический блок 3.1. Импорты и рабочие каталоги

```
from pathlib import Path
from time import perf_counter
import csv
import json
import math
import zipfile

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

from pyspark.sql import functions as F, Window
from pyspark.sql.types import (
    StructType, StructField, LongType, IntegerType,
    DoubleType, StringType, BooleanType
)
from pyspark.ml import Pipeline, PipelineModel
from pyspark.ml.functions import vector_to_array
from pyspark.ml.feature import (
    SQLTransformer, StringIndexer, OneHotEncoder,
    VectorAssembler, StandardScaler, PolynomialExpansion, PCA, Imputer
)
from pyspark.ml.classification import LogisticRegression, RandomForestClassifier
from pyspark.ml.regression import LinearRegression, RandomForestRegressor, GBTRegressor
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import (
    BinaryClassificationEvaluator, RegressionEvaluator, ClusteringEvaluator
)
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

from sklearn.metrics import (
    precision_recall_curve, roc_curve, confusion_matrix,
    silhouette_score
)

pd.set_option("display.width", 160)
pd.set_option("display.max_columns", 30)

# Единый стиль графиков: фиксированные цвета сущностей, приглушённая сетка
C1, C2, C3 = "#2a78d6", "#eb6834", "#1baf7a"   # модель 1 / модель 2 / модель 3
GRAY = "#8a8985"                               # базовые правила и вспомогательные линии
PALETTE = [C1, C2, C3, "#eda100", "#e87ba4", "#008300", "#6250d6", "#e34948"]
plt.rcParams.update({
    "figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False,
    "axes.grid": True, "grid.alpha": 0.25, "axes.titleweight": "bold",
})

SEED = 42
ROOT = Path.cwd()
WORK = ROOT / "practice5_data"
OUT = ROOT / "practice5_results"
WORK.mkdir(exist_ok=True)
OUT.mkdir(exist_ok=True)

SHOP_SOURCE = ROOT / "online_shoppers_intention.csv"
BIKE_SOURCE = ROOT / "SeoulBikeData.csv"
```

```
for path in [SHOP_SOURCE, BIKE_SOURCE]:
    if not path.is_file():
        raise FileNotFoundError(f"Не найден файл: {path}")

sc = spark.sparkContext
sc.setLogLevel("WARN")
print("Spark:", spark.version)
print("Master:", sc.master)
print("Spark UI:", sc.uiWebUrl)
```

```
Spark: 3.5.3
Master: local[*]
Spark UI: http://host.docker.internal:4040
```

Переменная `sc` указывает на работающий SparkContext. Адрес Spark UI выводится фактически: порт может отличаться от 4040.

Практический блок 3.2. Устойчивый row_id для интернет-сессий

`row_id` повторяет исходный порядок CSV и не входит в features. Он нужен для воспроизводимого разбиения, сохранения предсказаний и соединения кластерных меток с исходной таблицей.

```
SHOP_WORK = WORK / "online_shoppers_with_id.csv"

with SHOP_SOURCE.open("r", encoding="utf-8-sig", newline="") as src, \
     SHOP_WORK.open("w", encoding="utf-8", newline="") as dst:
    reader = csv.reader(src)
    writer = csv.writer(dst)
    header = next(reader)
    writer.writerow(["row_id"] + header)
    for row_id, row in enumerate(reader, start=1):
        writer.writerow([row_id] + row)

print("Исходный заголовок:", header)
```

```
Исходный заголовок: ['Administrative', 'Administrative_Duration', 'Informational', 'Informational_Duration',
'ProductRelated', 'ProductRelated_Duration', 'BounceRates', 'ExitRates', 'PageValues', 'SpecialDay',
'Month', 'OperatingSystems', 'Browser', 'Region', 'TrafficType', 'VisitorType', 'Weekend', 'Revenue']
```

Практический блок 3.3. Явные схемы и чтение CSV

```
shop_schema = StructType([
    StructField("row_id", LongType(), False),
    StructField("Administrative", DoubleType(), False),
    StructField("Administrative_Duration", DoubleType(), False),
    StructField("Informational", DoubleType(), False),
    StructField("Informational_Duration", DoubleType(), False),
    StructField("ProductRelated", DoubleType(), False),
    StructField("ProductRelated_Duration", DoubleType(), False),
    StructField("BounceRates", DoubleType(), False),
    StructField("ExitRates", DoubleType(), False),
    StructField("PageValues", DoubleType(), False),
    StructField("SpecialDay", DoubleType(), False),
    StructField("Month", StringType(), False),
    StructField("OperatingSystems", IntegerType(), False),
    StructField("Browser", IntegerType(), False),
    StructField("Region", IntegerType(), False),
    StructField("TrafficType", IntegerType(), False),
    StructField("VisitorType", StringType(), False),
    StructField("Weekend", BooleanType(), False),
    StructField("Revenue", BooleanType(), False),
])

bike_schema = StructType([
```

```
    StructField("date_text", StringType(), False),
    StructField("label", DoubleType(), False),
    StructField("hour", IntegerType(), False),
    StructField("temperature", DoubleType(), False),
    StructField("humidity", DoubleType(), False),
    StructField("wind_speed", DoubleType(), False),
    StructField("visibility", DoubleType(), False),
    StructField("dew_point", DoubleType(), False),
    StructField("solar_radiation", DoubleType(), False),
    StructField("rainfall", DoubleType(), False),
    StructField("snowfall", DoubleType(), False),
    StructField("season", StringType(), False),
    StructField("holiday", StringType(), False),
    StructField("functioning_day", StringType(), False),
])

shop_raw = (spark.read
    .option("header", True)
    .option("mode", "FAILFAST")
    .schema(shop_schema)
    .csv(SHOP_WORK.resolve().as_posix()))

bike_raw = (spark.read
    .option("header", True)
    .option("encoding", "ISO-8859-1")
    .option("enforceSchema", True)
    .option("mode", "FAILFAST")
    .schema(bike_schema)
    .csv(BIKE_SOURCE.resolve().as_posix()))

# Контроль порядка столбцов Seoul Bike: исходный заголовок против назначенных имён
with BIKE_SOURCE.open("r", encoding="ISO-8859-1") as f:
    bike_header = next(csv.reader(f))
for original, safe in zip(bike_header, bike_schema.fieldNames()):
    print(f"{original:30s} -> {safe}")

print("\nOnline Shoppers:", shop_raw.count(), "строк")
print("Seoul Bike:", bike_raw.count(), "строк")
shop_raw.printSchema()
bike_raw.printSchema()
```

```
Date                           -> date_text
Rented Bike Count              -> label
Hour                           -> hour
Temperature(°C)                -> temperature
Humidity(%)                    -> humidity
Wind speed (m/s)               -> wind_speed
Visibility (10m)               -> visibility
Dew point temperature(°C)      -> dew_point
Solar Radiation (MJ/m2)        -> solar_radiation
Rainfall(mm)                   -> rainfall
Snowfall (cm)                  -> snowfall
Seasons                        -> season
Holiday                        -> holiday
Functioning Day                -> functioning_day
Online Shoppers: 12330 строк
```

```
Seoul Bike: 8760 строк
root
 |-- row_id: long (nullable = true)
 |-- Administrative: double (nullable = true)
 |-- Administrative_Duration: double (nullable = true)
 |-- Informational: double (nullable = true)
 |-- Informational_Duration: double (nullable = true)
 |-- ProductRelated: double (nullable = true)
 |-- ProductRelated_Duration: double (nullable = true)
 |-- BounceRates: double (nullable = true)
 |-- ExitRates: double (nullable = true)
 |-- PageValues: double (nullable = true)
 |-- SpecialDay: double (nullable = true)
 |-- Month: string (nullable = true)
 |-- OperatingSystems: integer (nullable = true)
 |-- Browser: integer (nullable = true)
 |-- Region: integer (nullable = true)
 |-- TrafficType: integer (nullable = true)
 |-- VisitorType: string (nullable = true)
 |-- Weekend: boolean (nullable = true)
 |-- Revenue: boolean (nullable = true)

root
 |-- date_text: string (nullable = true)
 |-- label: double (nullable = true)
 |-- hour: integer (nullable = true)
 |-- temperature: double (nullable = true)
 |-- humidity: double (nullable = true)
 |-- wind_speed: double (nullable = true)
 |-- visibility: double (nullable = true)
 |-- dew_point: double (nullable = true)
 |-- solar_radiation: double (nullable = true)
 |-- rainfall: double (nullable = true)
 |-- snowfall: double (nullable = true)
 |-- season: string (nullable = true)
 |-- holiday: string (nullable = true)
 |-- functioning_day: string (nullable = true)
```

Файл `SeoulBikeData.csv` использует однобайтовую кодировку для символа градуса в заголовке, поэтому чтение выполняется с `encoding="ISO-8859-1"`. Безопасные имена схемы назначаются по порядку столбцов — выше выведено соответствие исходного заголовка и новых имён.

Практический блок 3.4. Контроль строк, ключей и пропусков

```python
def null_counts(df):
    return df.select(*[
        F.sum(F.col(c).isNull().cast("int")).alias(c)
        for c in df.columns
    ]).first().asDict()

print("Пропуски Online Shoppers:", {k: v for k, v in null_counts(shop_raw).items() if v} or "нет")
print("Пропуски Seoul Bike     :", {k: v for k, v in null_counts(bike_raw).items() if v} or "нет")
print("Уникальных row_id:", shop_raw.select("row_id").distinct().count(), "из", shop_raw.count())
print("Полных дублей сессий (без row_id):",
      shop_raw.count() - shop_raw.drop("row_id").distinct().count())
print("Уникальных пар (дата, час) в Seoul Bike:",
      bike_raw.select("date_text", "hour").distinct().count())

shop_raw.groupBy("Revenue").count().orderBy("Revenue").show()
bike_raw.groupBy("functioning_day").agg(
    F.count("*").alias("n"),
    F.min("label").alias("min_target"),
    F.max("label").alias("max_target")
).show()
```

```
Пропуски Online Shoppers: нет
Пропуски Seoul Bike     : нет
Уникальных row_id: 12330 из 12330
Полных дублей сессий (без row_id): 125
Уникальных пар (дата, час) в Seoul Bike: 8760
+-------+-----+
|Revenue|count|
+-------+-----+
|  false|10422|
|   true| 1908|
+-------+-----+

+---------------+----+----------+----------+
|functioning_day|   n|min_target|max_target|
+---------------+----+----------+----------+
|             No| 295|       0.0|       0.0|
|            Yes|8465|       2.0|    3556.0|
+---------------+----+----------+----------+
```

Результат контроля. Online Shoppers — 12 330 строк, Seoul Bike — 8 760 строк, пропусков нет ни в одном наборе; `row_id` уникален, пары (дата, час) в Seoul Bike тоже уникальны. Покупкой заканчиваются 1 908 сессий из 12 330 (15,5 %). В Online Shoppers найдено 125 строк, полностью совпадающих с другими по всем полям: идентификатора клиента нет, поэтому это могут быть как разные, но одинаковые по признакам короткие сессии, так и повторы — в основном сценарии они, как в методичке, сохраняются (это ограничение данных). 295 часов `Functioning Day = No` имеют ровно 0 аренд — это закрытие сервиса, а не низкий спрос; основная регрессия отвечает на вопрос «сколько аренд, когда сервис работает».

2. Классификация покупательского намерения (Online Shoppers)

4.1. Постановка и дисбаланс классов

Цель — оценить вероятность `Revenue = True`. Положительный класс редкий, поэтому модели сравниваются прежде всего по PR-AUC, precision, recall и F1, а не только по accuracy.

```
cls = shop_raw.groupBy("Revenue").count().orderBy("Revenue").toPandas()
total = cls["count"].sum()
fig, ax = plt.subplots(figsize=(5.6, 3.8))
bars = ax.bar(["Без покупки", "Покупка"], cls["count"], color=[GRAY, C1], width=0.55)
for b, n in zip(bars, cls["count"]):
    ax.text(b.get_x() + b.get_width() / 2, n + 120, f"{n:,} ({n / total:.1%})".replace(",", " "),
            ha="center", va="bottom", fontsize=10)
ax.set(ylabel="Число сессий", title="Рис. 4. Распределение классов Online Shoppers",
       ylim=(0, cls["count"].max() * 1.12))
ax.grid(axis="x", visible=False)
plt.tight_layout()
plt.savefig(OUT / "classification_classes.png", dpi=160)
plt.show()
print(f"Accuracy постоянного ответа «нет покупки»: {cls['count'].iloc[0] / total:.3f}, recall покупки = 0")
```




```
Accuracy постоянного ответа «нет покупки»: 0.845, recall покупки = 0
```

4.2–4.3. Момент доступности признаков, разбиение и веса классов

`PageValues` включён в основной опыт (оценка по накопленному состоянию сессии), а затем проверяется абляцией. Шесть счётчиков и длительностей логарифмируются внутри Pipeline; `OperatingSystems`, `Browser`, `Region`, `TrafficType` рассматриваются как категории. Разбиение 60/20/20 выполняется отдельно внутри каждого класса по SHA-256 от `row_id` и seed, веса классов $w_c = N/(K \cdot N_c)$ считаются только по train.

```
SHOP_LOG_SOURCE = [
    "Administrative", "Administrative_Duration",
```

```
    "Informational", "Informational_Duration",
    "ProductRelated", "ProductRelated_Duration"
]
SHOP_LOG = ["log_" + c for c in SHOP_LOG_SOURCE]
SHOP_NUM_BASE = SHOP_LOG + ["BounceRates", "ExitRates", "SpecialDay"]
SHOP_CAT = [
    "Month", "OperatingSystems", "Browser", "Region",
    "TrafficType", "VisitorType", "Weekend"
]

shop = shop_raw.withColumn("label", F.col("Revenue").cast("double"))
for c in ["OperatingSystems", "Browser", "Region", "TrafficType", "Weekend"]:
    shop = shop.withColumn(c, F.col(c).cast("string"))

order_key = F.sha2(F.concat_ws(
    ":", F.col("row_id").cast("string"), F.lit(str(SEED))
), 256)
rank_window = Window.partitionBy("label").orderBy(order_key, "row_id")
class_window = Window.partitionBy("label")

shop_split = (shop
    .withColumn("_rank", F.row_number().over(rank_window))
    .withColumn("_size", F.count("*").over(class_window))
    .withColumn(
        "split",
        F.when(F.col("_rank") <= F.floor(F.col("_size") * 0.60), "train")
         .when(F.col("_rank") <= F.floor(F.col("_size") * 0.80), "valid")
         .otherwise("test")
    )
    .drop("_rank", "_size")
    .cache())

split_table = (shop_split.groupBy("split").pivot("label", [0.0, 1.0]).count()
    .withColumnRenamed("0.0", "no_purchase").withColumnRenamed("1.0", "purchase")
    .withColumn("total", F.col("no_purchase") + F.col("purchase"))
    .withColumn("purchase_share", F.round(F.col("purchase") / F.col("total"), 4))
    .orderBy(F.expr("CASE split WHEN 'train' THEN 0 WHEN 'valid' THEN 1 ELSE 2 END")))
split_table.show()

shop_train = shop_split.filter("split = 'train'").drop("split")
shop_valid = shop_split.filter("split = 'valid'").drop("split")
shop_test = shop_split.filter("split = 'test'").drop("split")

counts = {int(r["label"]): r["count"]
          for r in shop_train.groupBy("label").count().collect()}
train_n = sum(counts.values())
weight_0 = train_n / (2.0 * counts[0])
weight_1 = train_n / (2.0 * counts[1])
shop_train = shop_train.withColumn(
    "class_weight",
    F.when(F.col("label") == 1.0, F.lit(weight_1)).otherwise(F.lit(weight_0))
)
print("Веса классов: w0 = %.4f, w1 = %.4f" % (weight_0, weight_1))
```

```
+-----+-----------+--------+-----+--------------+
|split|no_purchase|purchase|total|purchase_share|
+-----+-----------+--------+-----+--------------+
|train|       6253|    1144| 7397|        0.1547|
|valid|       2084|     382| 2466|        0.1549|
| test|       2085|     382| 2467|        0.1548|
+-----+-----------+--------+-----+--------------+

Веса классов: w0 = 0.5915, w1 = 3.2330
```

Состав частей совпадает с контрольной таблицей 4 методички: train 6 253 / 1 144, valid 2 084 / 382, test 2 085 / 382; доля покупок во всех частях ≈ 15,5 %. Веса классов по train: w_0 = 0,59, w_1 = 3,23 —

ошибка на покупке штрафуется примерно в 5,5 раза сильнее, число строк при этом не меняется.

Практический блок 4.5. Pipeline двух классификаторов

`SQLTransformer` создаёт логарифмированные поля; `StringIndexer` обучает словари категорий на train; `OneHotEncoder` превращает индексы в индикаторы; `VectorAssembler` объединяет координаты. В ветке LR вектор масштабируется (`withMean=False` , чтобы не уплотнять разреженный вектор) и передаётся в `LogisticRegression` с L2 и весами классов; в ветке RF масштабирование не нужно.

```
def shop_pipeline(kind, include_page_values=True):
    expressions = [
        f"log1p(`{source}`) AS `{target}`"
        for source, target in zip(SHOP_LOG_SOURCE, SHOP_LOG)
    ]
    log_stage = SQLTransformer(
        statement="SELECT *, " + ", ".join(expressions) + " FROM __THIS__"
    )

    index_cols = [c + "_idx" for c in SHOP_CAT]
    encoded_cols = [c + "_ohe" for c in SHOP_CAT]
    indexer = StringIndexer(
        inputCols=SHOP_CAT,
        outputCols=index_cols,
        handleInvalid="keep",
        stringOrderType="alphabetAsc"
    )
    encoder = OneHotEncoder(
        inputCols=index_cols,
        outputCols=encoded_cols,
        handleInvalid="keep",
        dropLast=False
    )

    numeric = SHOP_NUM_BASE + (["PageValues"] if include_page_values else [])
    if kind == "LR":
        assembler = VectorAssembler(
            inputCols=numeric + encoded_cols,
            outputCol="features_raw"
        )
        scaler = StandardScaler(
            inputCol="features_raw", outputCol="features",
            withMean=False, withStd=True
        )
        estimator = LogisticRegression(
            maxIter=100, regParam=0.05, elasticNetParam=0.0,
            standardization=False, weightCol="class_weight"
        )
        stages = [log_stage, indexer, encoder, assembler, scaler, estimator]
    elif kind == "RF":
        assembler = VectorAssembler(
            inputCols=numeric + encoded_cols,
            outputCol="features"
        )
        estimator = RandomForestClassifier(
            numTrees=80, maxDepth=8, maxBins=64,
            featureSubsetStrategy="sqrt", seed=SEED,
            weightCol="class_weight"
        )
        stages = [log_stage, indexer, encoder, assembler, estimator]
    else:
        raise ValueError(kind)
    return Pipeline(stages=stages)
```

```
def binary_scores(model, df):
    return (model.transform(df)
        .withColumn("score", vector_to_array("probability")[1])
        .select("row_id", "label", "score"))


def binary_metrics(pred, threshold=0.5, ranking=True):
    decision = F.col("score") >= F.lit(float(threshold))
    y = F.col("label") == 1.0
    values = pred.agg(
        F.sum((decision & y).cast("long")).alias("tp"),
        F.sum((decision & ~y).cast("long")).alias("fp"),
        F.sum((~decision & y).cast("long")).alias("fn"),
        F.sum((~decision & ~y).cast("long")).alias("tn")
    ).first().asDict()

    tp, fp, fn, tn = (values[k] for k in ["tp", "fp", "fn", "tn"])
    n = tp + fp + fn + tn
    precision = tp / (tp + fp) if tp + fp else 0.0
    recall = tp / (tp + fn) if tp + fn else 0.0
    f1 = 2 * precision * recall / (precision + recall) if precision + recall else 0.0
    result = {
        "accuracy": (tp + tn) / n,
        "precision": precision,
        "recall": recall,
        "f1": f1,
        "tp": tp, "fp": fp, "fn": fn, "tn": tn
    }
    if ranking:
        for metric in ["areaUnderROC", "areaUnderPR"]:
            result[metric] = BinaryClassificationEvaluator(
                labelCol="label", rawPredictionCol="score", metricName=metric
            ).evaluate(pred)
    return result


def show_metrics(title, m):
    print(f"{title}: accuracy={m['accuracy']:.3f}; precision={m['precision']:.3f}; "
          f"recall={m['recall']:.3f}; F1={m['f1']:.3f}"
          + (f"; ROC-AUC={m['areaUnderROC']:.3f}; PR-AUC={m['areaUnderPR']:.3f}" if "areaUnderPR" in m else
""))
    print(f"   Матрица ошибок: TN={m['tn']}, FP={m['fp']}, FN={m['fn']}, TP={m['tp']}")
```

4.6. Метрики и обучение

Обе полные цепочки обучаются на одном `shop_train`; `setJobGroup` даёт понятные подписи в Spark UI, `perf_counter` измеряет время `fit`. Победитель выбирается по `areaUnderPR` на valid, а не по test.

```
shop_models = {}
shop_valid_predictions = {}
shop_rows = []

for name, kind in [("Логистическая регрессия", "LR"),
                   ("Случайный лес", "RF")]:
    sc.setJobGroup("P5_CLASS_" + kind, "Классификация: " + name)
    t0 = perf_counter()
    model = shop_pipeline(kind, include_page_values=True).fit(shop_train)
    fit_s = perf_counter() - t0
    pred = binary_scores(model, shop_valid).cache()
    pred.count()
    row = {"model": name, "fit_s": fit_s, **binary_metrics(pred)}
    shop_models[name] = model
```

```
    shop_valid_predictions[name] = pred
    shop_rows.append(row)

shop_validation = pd.DataFrame(shop_rows).sort_values("areaUnderPR", ascending=False)
print("Таблица 5. Результат на valid при пороге 0,5")
print(shop_validation.round(4).to_string(index=False))
BEST_SHOP_NAME = str(shop_validation.iloc[0]["model"])
BEST_SHOP = shop_models[BEST_SHOP_NAME]
print("\nЛучшая модель по PR-AUC на valid:", BEST_SHOP_NAME)
```

```
Таблица 5. Результат на valid при пороге 0,5
                  model   fit_s  accuracy  precision  recall     f1  tp  fp  fn   tn  areaUnderROC
areaUnderPR
          Случайный лес 15.1359    0.8759     0.5660  0.8534 0.6806 326 250  56 1834        0.9299
0.7167
Логистическая регрессия 23.8870    0.8204     0.4529  0.7670 0.5695 293 354  89 1730        0.8950
0.6102

Лучшая модель по PR-AUC на valid: Случайный лес
```

Результат (valid, порог 0,5). Случайный лес лучше по всем ранжирующим и пороговым метрикам: PR-AUC 0,717 против 0,610, ROC-AUC 0,930 против 0,895, F1 0,681 против 0,570. Обе модели благодаря весам классов находят большую часть покупок (recall 0,85 и 0,77), но платят ложными срабатываниями: у леса 250 FP на 326 TP (precision 0,57), у LR — 354 FP на 293 TP (precision 0,45). По правилу «победитель по PR-AUC на valid» выбран случайный лес. Значения близки к контрольной таблице 5 (0,711 / 0,634); отличия объясняются другой версией Spark и случайностью деревьев.

4.7. Выбор порога и однократный test

Порог выбирается только на valid — в максимуме F1 по `precision_recall_curve`. Таблица valid выбранной модели (≈2,5 тыс. строк, два столбца) безопасно переносится в pandas. После фиксации порога test используется один раз.

```
valid_pd = shop_valid_predictions[BEST_SHOP_NAME].toPandas()
precision, recall, thresholds = precision_recall_curve(
    valid_pd["label"], valid_pd["score"]
)
f1_values = 2 * precision * recall / np.maximum(precision + recall, 1e-12)
best_index = int(np.nanargmax(f1_values[:-1]))
SHOP_THRESHOLD = float(thresholds[best_index])
print("Выбранная модель:", BEST_SHOP_NAME)
print("Порог по максимуму F1 на valid:", round(SHOP_THRESHOLD, 4))
shop_valid_best = binary_metrics(shop_valid_predictions[BEST_SHOP_NAME], SHOP_THRESHOLD, ranking=True)
show_metrics("VALID (выбранный порог)", shop_valid_best)

sc.setJobGroup("P5_CLASS_TEST", "Классификация: однократная проверка на test")
shop_test_scores = binary_scores(BEST_SHOP, shop_test).cache()
shop_test_scores.count()
shop_test_metrics = binary_metrics(shop_test_scores, SHOP_THRESHOLD, ranking=True)
show_metrics("TEST", shop_test_metrics)
```

```
Выбранная модель: Случайный лес
Порог по максимуму F1 на valid: 0.5384
VALID (выбранный порог): accuracy=0.882; precision=0.585; recall=0.819; F1=0.683; ROC-AUC=0.930; PR-
AUC=0.717
   Матрица ошибок: TN=1862, FP=222, FN=69, TP=313
TEST: accuracy=0.875; precision=0.571; recall=0.764; F1=0.654; ROC-AUC=0.910; PR-AUC=0.679
   Матрица ошибок: TN=1866, FP=219, FN=90, TP=292
```

Порог и итоговый test. Максимум F1 на valid достигается при пороге 0,538 (в методичке ≈ 0,55): F1 = 0,683, recall 0,819, precision 0,585. На test с тем же порогом: accuracy 0,875; precision 0,571; recall 0,764; F1 0,654; ROC-AUC 0,910; PR-AUC 0,679. Модель находит 292 из 382 покупок (пропускает 90), а

219 сессий без покупки получают положительное решение. Снижение F1 с 0,683 до 0,654 ожидаемо: порог подобран на valid и для новых данных немного оптимистичен. Для сравнения: accuracy 0,875 лишь на 3 п.п. выше постоянного ответа «нет покупки» (0,845), у которого recall = 0.

4.8. Кривые качества

Кривые строятся по небольшим локальным таблицам `score` / `label`, обучение и предсказания остаются в Spark. Дополнительно к методичке построен график precision / recall / F1 в зависимости от порога — он показывает, где именно был зафиксирован порог.

```
MODEL_COLORS = {"Логистическая регрессия": C1, "Случайный лес": C2}
local_valid = {name: shop_valid_predictions[name].toPandas() for name in MODEL_COLORS}

# Рис. 8. Precision–Recall
fig, ax = plt.subplots(figsize=(7.5, 4.8))
for name, color in MODEL_COLORS.items():
    p, r, _ = precision_recall_curve(local_valid[name]["label"], local_valid[name]["score"])
    ax.plot(r, p, label=name, color=color, linewidth=2)
ax.axhline(valid_pd["label"].mean(), linestyle="--", color=GRAY,
           label="Доля положительного класса")
ax.set(xlabel="Recall", ylabel="Precision", title="Рис. 8. Precision–Recall-кривые на valid")
ax.legend()
plt.tight_layout(); plt.savefig(OUT / "classification_pr.png", dpi=160); plt.show()

# Рис. 9. ROC
fig, ax = plt.subplots(figsize=(6.2, 5.2))
for name, color in MODEL_COLORS.items():
    fpr, tpr, _ = roc_curve(local_valid[name]["label"], local_valid[name]["score"])
    ax.plot(fpr, tpr, label=name, color=color, linewidth=2)
ax.plot([0, 1], [0, 1], "--", color=GRAY, label="Случайное ранжирование")
ax.set(xlabel="False positive rate", ylabel="True positive rate", title="Рис. 9. ROC-кривые на valid")
ax.legend(loc="lower right")
plt.tight_layout(); plt.savefig(OUT / "classification_roc.png", dpi=160); plt.show()

# Дополнительно: метрики в зависимости от порога (valid, выбранная модель)
fig, ax = plt.subplots(figsize=(7.5, 4.2))
ax.plot(thresholds, precision[:-1], color=C1, linewidth=2, label="Precision")
ax.plot(thresholds, recall[:-1], color=C2, linewidth=2, label="Recall")
ax.plot(thresholds, f1_values[:-1], color=C3, linewidth=2, label="F1")
ax.axvline(SHOP_THRESHOLD, color=GRAY, linestyle="--")
ax.text(SHOP_THRESHOLD + 0.01, 0.05, f"порог = {SHOP_THRESHOLD:.3f}", color="#52514e")
ax.set(xlabel="Порог score", ylabel="Значение метрики", ylim=(0, 1.02),
       title=f"Метрики valid в зависимости от порога ({BEST_SHOP_NAME})")
ax.legend(loc="center left")
plt.tight_layout(); plt.savefig(OUT / "classification_threshold.png", dpi=160); plt.show()

# Рис. 7. Матрица ошибок на test
final_pd = shop_test_scores.toPandas()
final_pred = (final_pd["score"] >= SHOP_THRESHOLD).astype(int)
cm = confusion_matrix(final_pd["label"].astype(int), final_pred, labels=[0, 1])
fig, ax = plt.subplots(figsize=(5.2, 4.2))
im = ax.imshow(cm, cmap="Blues")
for i in range(2):
    for j in range(2):
        ax.text(j, i, cm[i, j], ha="center", va="center", fontsize=13,
                color="white" if cm[i, j] > cm.max() / 2 else "#0b0b0b")
ax.set_xticks([0, 1], ["Нет покупки", "Покупка"])
ax.set_yticks([0, 1], ["Нет покупки", "Покупка"])
ax.grid(False)
ax.set(xlabel="Решение модели", ylabel="Факт", title="Рис. 7. Матрица ошибок на test")
fig.colorbar(im, ax=ax)
plt.tight_layout(); plt.savefig(OUT / "classification_confusion.png", dpi=160); plt.show()
```

Рис. 8. Precision–Recall-кривые на valid

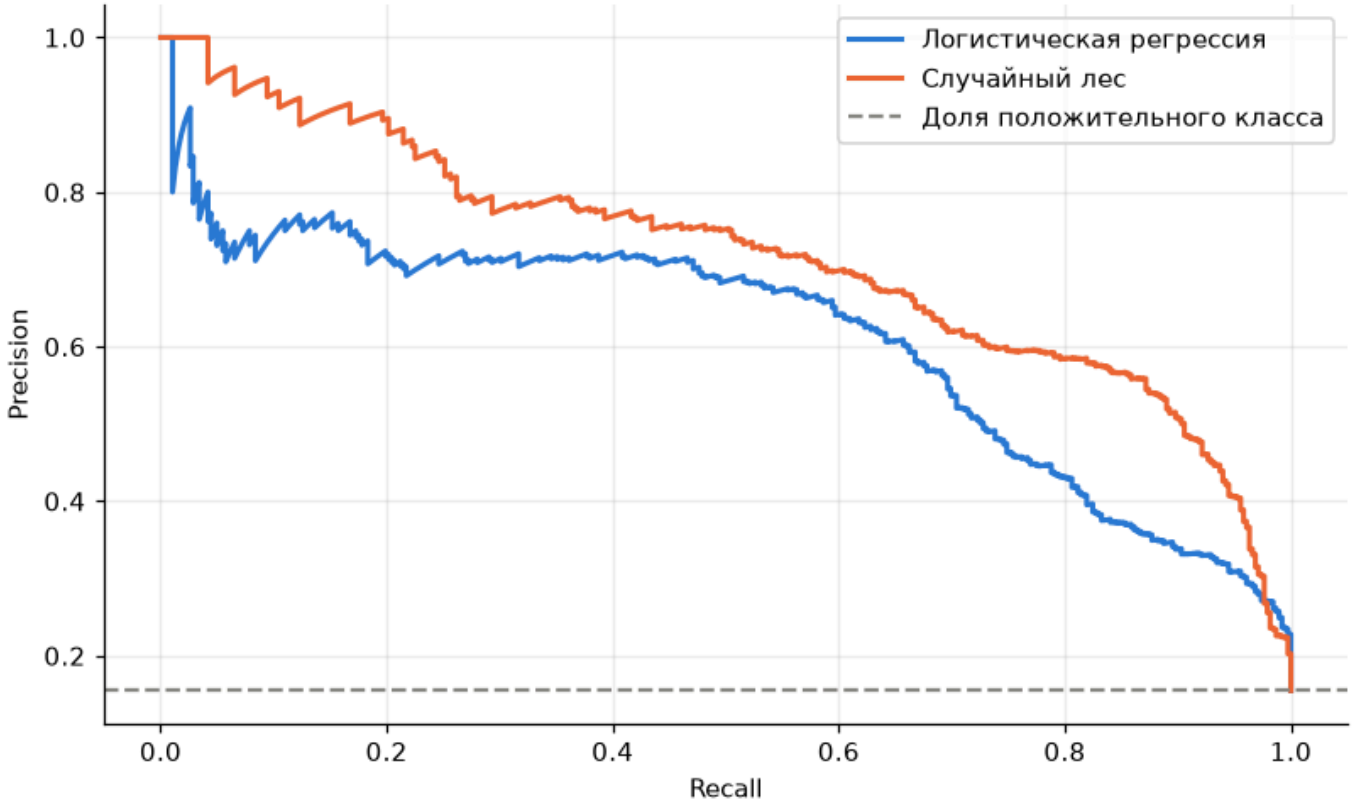
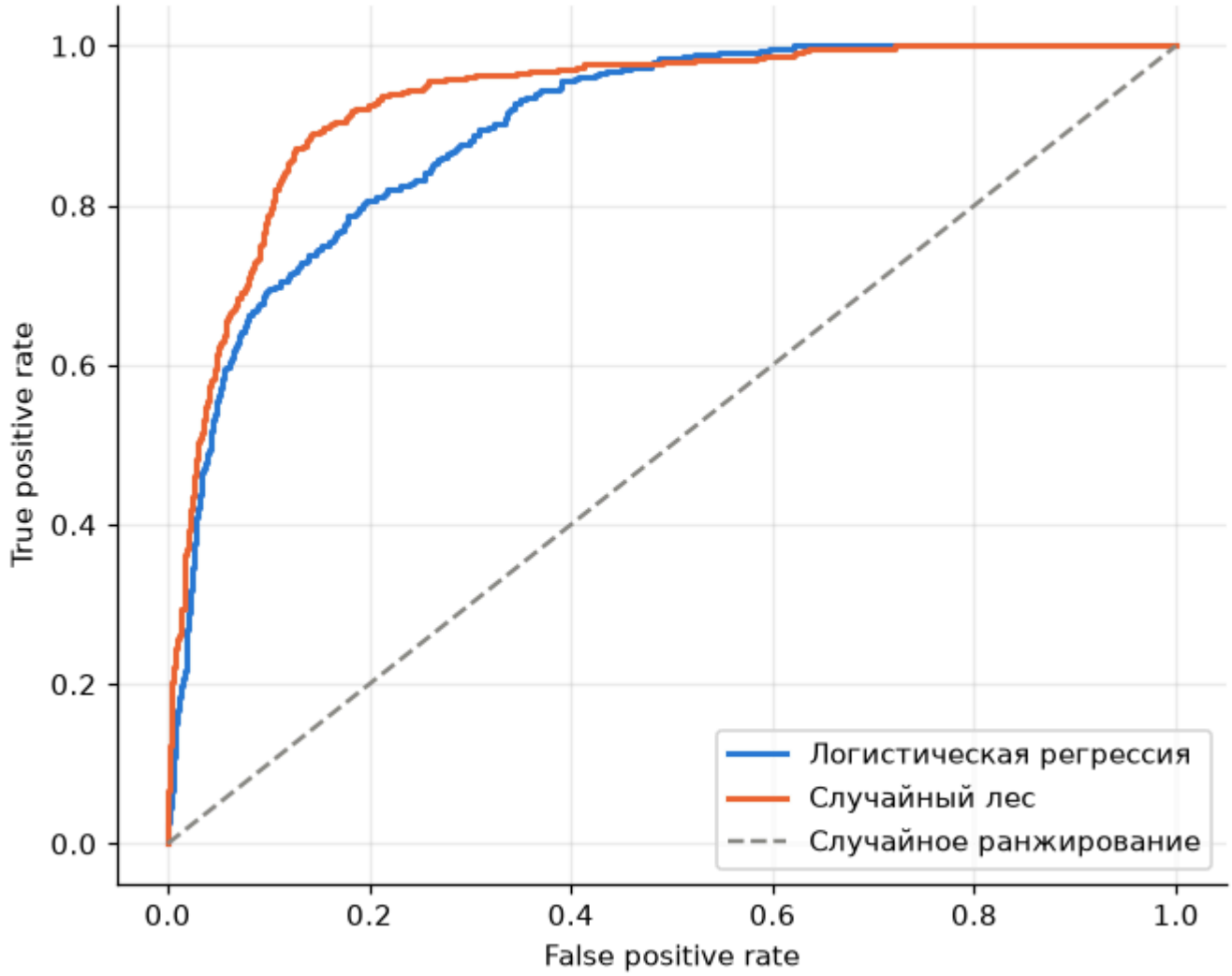
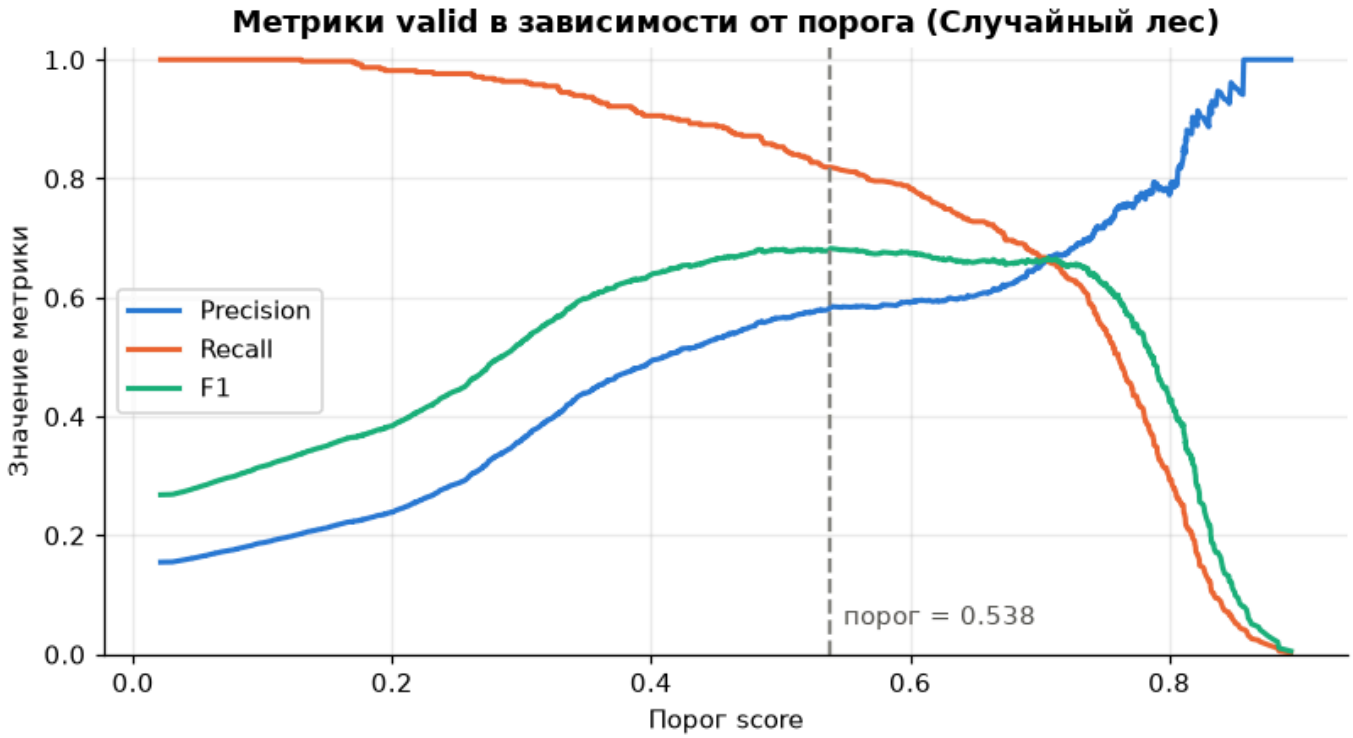
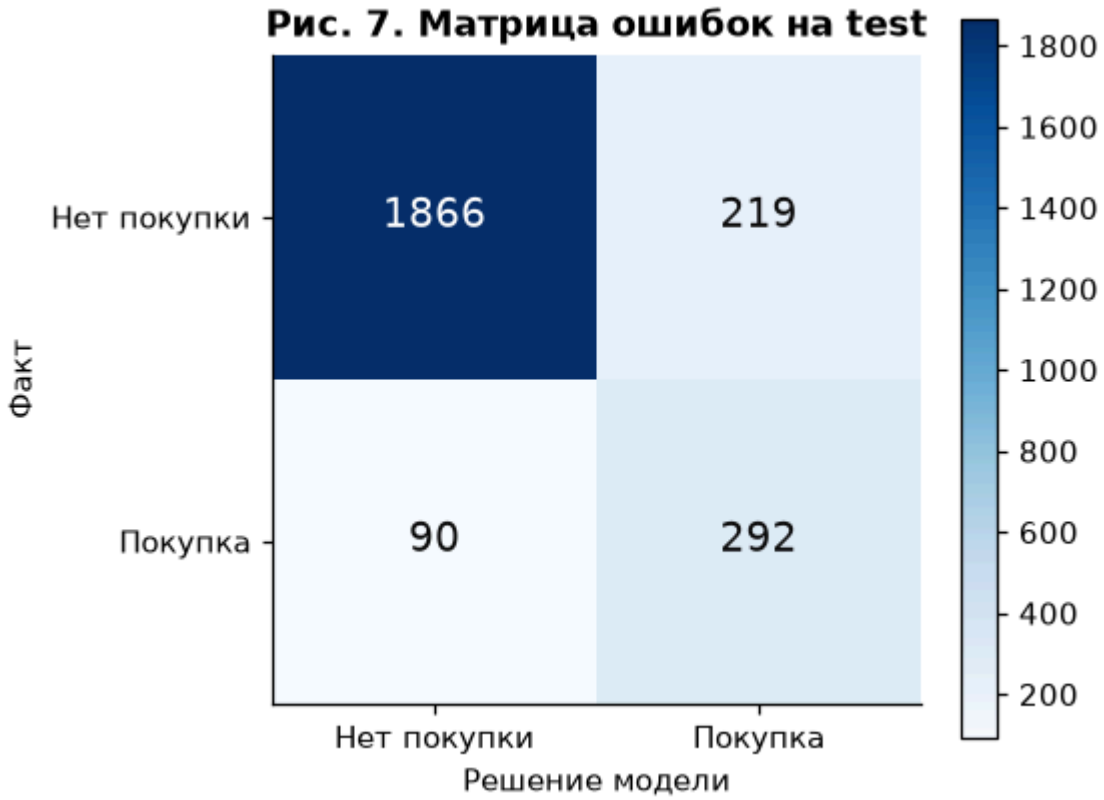


Рис. 9. ROC-кривые на valid







Чтение кривых. PR-кривая леса почти везде лежит выше кривой LR и намного выше пунктира доли покупок (0,155) — ранжирование содержательно лучше случайного. ROC-кривые обеих моделей близки к левому верхнему углу, но из-за дисбаланса ROC-AUC выглядит оптимистичнее PR-AUC: значение 0,93 соседствует с ~220 ложными срабатываниями. На графике метрик от порога видно, что F1 имеет пологий максимум около 0,5–0,6: небольшие сдвиги порога мало меняют F1, но заметно перераспределяют precision и recall. В матрице ошибок test правая верхняя ячейка (FP = 219) и левая нижняя (FN = 90) — две разные цены ошибки: лишнее предложение против пропущенной покупки.

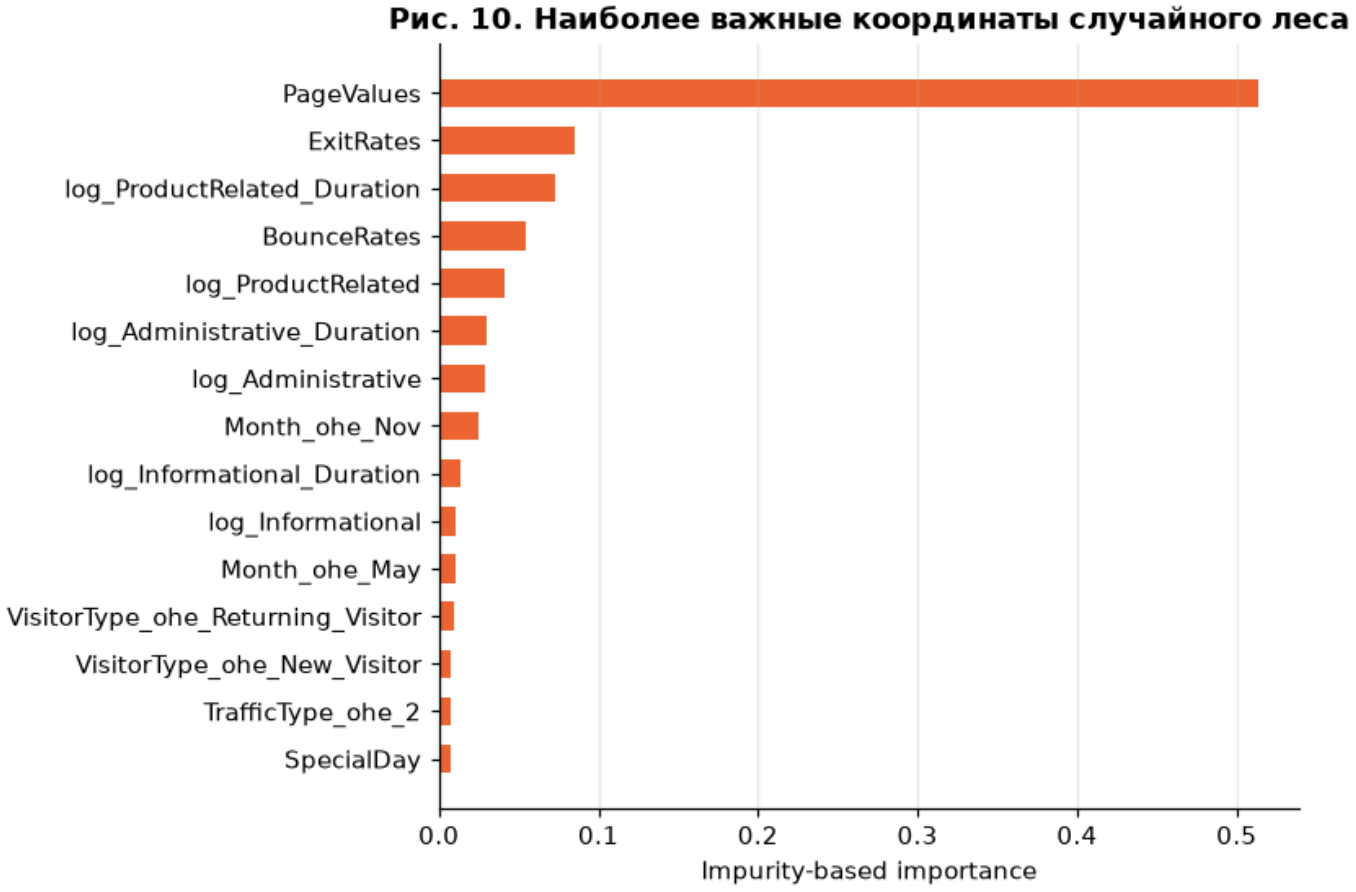
4.9. Важность признаков, коэффициенты LR и абляция PageValues

Имена координат подготовленного вектора берутся из метаданных `VectorAssembler` (`ml_attr`), поэтому каждая one-hot-координата получает читаемое имя вида `Month_ohe_Nov`.

```
def vector_names(model, df, col):
    attrs = model.transform(df.limit(1)).schema[col].metadata["ml_attr"]["attrs"]
    names = {}
    for group in attrs.values():
        for a in group:
            names[a["idx"]] = a["name"]
    return [names[i] for i in range(len(names))]

rf_shop = shop_models["Случайный лес"]
rf_names = vector_names(rf_shop, shop_valid, "features")
rf_imp = (pd.Series(rf_shop.stages[-1].featureImportances.toArray(), index=rf_names)
          .sort_values(ascending=False))
top = rf_imp.head(15)[::-1]
fig, ax = plt.subplots(figsize=(7.5, 5.2))
ax.barh(top.index, top.values, color=C2, height=0.6)
ax.set(xlabel="Impurity-based importance", title="Рис. 10. Наиболее важные координаты случайного леса")
ax.grid(axis="y", visible=False)
plt.tight_layout(); plt.savefig(OUT / "classification_importance.png", dpi=160); plt.show()
print(rf_imp.head(10).round(4).to_string())

lr_shop = shop_models["Логистическая регрессия"]
lr_names = vector_names(lr_shop, shop_valid, "features_raw")
lr_coef = pd.Series(lr_shop.stages[-1].coefficients.toArray(), index=lr_names)
print("\nLR: 8 наибольших и 8 наименьших коэффициентов (в масштабированных единицах):")
print(pd.concat([lr_coef.sort_values(ascending=False).head(8),
                 lr_coef.sort_values().head(8)]).round(3).to_string())
```

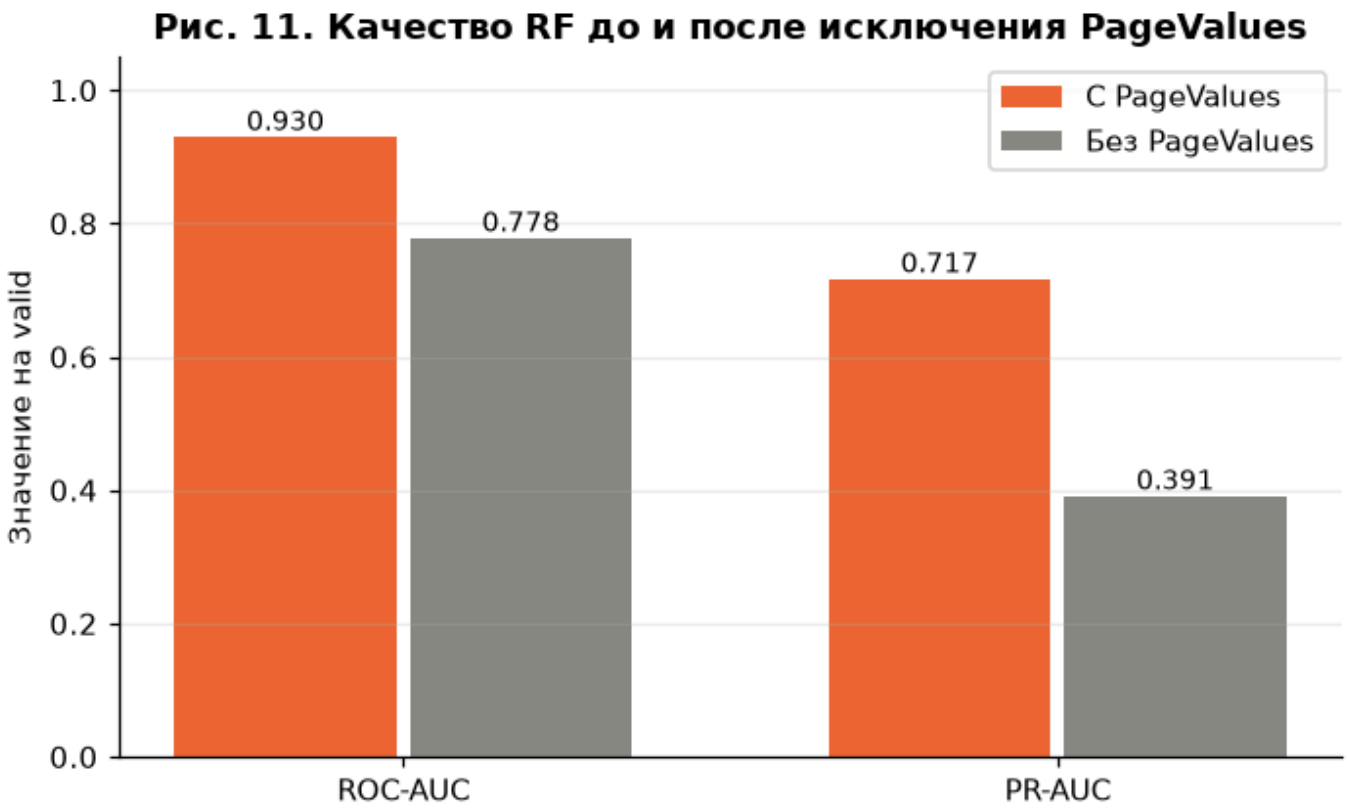


```
PageValues                     0.5134
ExitRates                      0.0847
log_ProductRelated_Duration    0.0732
BounceRates                    0.0547
log_ProductRelated             0.0415
log_Administrative_Duration    0.0299
log_Administrative             0.0291
Month_ohe_Nov                  0.0249
log_Informational_Duration     0.0133
log_Informational              0.0104
LR: 8 наибольших и 8 наименьших коэффициентов (в масштабированных единицах):
PageValues                     1.071
Month_ohe_Nov                  0.244
log_ProductRelated             0.177
log_ProductRelated_Duration    0.160
log_Administrative_Duration    0.095
TrafficType_ohe_8              0.084
log_Informational              0.081
Month_ohe_Sep                  0.079
ExitRates                     -0.224
Month_ohe_May                 -0.128
Month_ohe_Mar                 -0.121
TrafficType_ohe_13            -0.119
BounceRates                   -0.115
Browser_ohe_3                 -0.111
Month_ohe_Feb                 -0.080
Month_ohe_Dec                 -0.077
```

```python
# Проверка чувствительности к PageValues на той же valid и с теми же параметрами леса
sc.setJobGroup("P5_CLASS_ABLATION", "Классификация: RF без PageValues")
rf_without_page_values = shop_pipeline("RF", include_page_values=False).fit(shop_train)
without_scores = binary_scores(rf_without_page_values, shop_valid).cache()
with_m = binary_metrics(shop_valid_predictions["Случайный лес"], ranking=True)
without_m = binary_metrics(without_scores, ranking=True)
ablation = pd.DataFrame([
    {"Вариант": "С PageValues", "ROC-AUC valid": with_m["areaUnderROC"], "PR-AUC valid":
with_m["areaUnderPR"], "F1@0.5": with_m["f1"]},
    {"Вариант": "Без PageValues", "ROC-AUC valid": without_m["areaUnderROC"], "PR-AUC valid":
without_m["areaUnderPR"], "F1@0.5": without_m["f1"]},
])
print("Таблица 6. Влияние признака PageValues для случайного леса")
print(ablation.round(3).to_string(index=False))

fig, ax = plt.subplots(figsize=(6.5, 4))
x = np.arange(2); w = 0.36
for i, (label, color) in enumerate([("С PageValues", C2), ("Без PageValues", GRAY)]):
    vals = ablation.loc[ablation["Вариант"] == label, ["ROC-AUC valid", "PR-AUC valid"]].values[0]
    bars = ax.bar(x + (i - 0.5) * w, vals, w * 0.94, color=color, label=label)
    for b, v in zip(bars, vals):
        ax.text(b.get_x() + b.get_width() / 2, v + 0.01, f"{v:.3f}", ha="center", fontsize=9)
ax.set_xticks(x, ["ROC-AUC", "PR-AUC"])
ax.set(ylim=(0, 1.05), ylabel="Значение на valid", title="Рис. 11. Качество RF до и после исключения
PageValues")
ax.legend(); ax.grid(axis="x", visible=False)
plt.tight_layout(); plt.savefig(OUT / "classification_ablation.png", dpi=160); plt.show()
```

```
Таблица 6. Влияние признака PageValues для случайного леса
       Вариант  ROC-AUC valid  PR-AUC valid  F1@0.5
  С PageValues          0.930         0.717   0.681
Без PageValues          0.778         0.391   0.410
```



Важность и абляция. У леса `PageValues` забирает 51 % суммарной важности, далее `ExitRates` (8 %), длительность товарных просмотров (7 %) и `BounceRates` (5 %); из календарных признаков выделяется ноябрь. У логистической регрессии `PageValues` также имеет наибольший положительный коэффициент (+1,07 на одно σ), `ExitRates` и `BounceRates` — отрицательные: чем чаще пользователь уходит со страниц, тем ниже шансы покупки. Это ассоциации, а не причинные эффекты.

Без `PageValues` на той же valid и с теми же параметрами леса PR-AUC падает с 0,717 до 0,391, ROC-AUC — с 0,930 до 0,778, F1@0,5 — с 0,681 до 0,410 (контрольные значения методички 0,401 и 0,779). Итог сильно зависит от одного поля: если оно недоступно в момент раннего прогноза, качество основного опыта переносить нельзя — нужны другие ранние признаки или иная постановка.

3. Регрессия спроса Seoul Bike

5.1–5.2. Постановка, временные признаки и части выборки

Цель — число аренд за час при работающем сервисе. Часы `Functioning Day = No` исключаются до обучения. Разбиение хронологическое: до августа 2018 — train, август–сентябрь — valid, октябрь–ноябрь — test. Час и месяц кодируются парой sin/cos, `day_index` — число дней от начала набора.

```
bike = (bike_raw
    .withColumn("date", F.to_date("date_text", "dd/MM/yyyy"))
    .drop("date_text")
    .filter(F.col("functioning_day") == "Yes")
    .withColumn("weekday", F.dayofweek("date").cast("string"))
    .withColumn("hour_sin",
                F.sin(F.lit(2.0 * math.pi) * F.col("hour") / F.lit(24.0)))
    .withColumn("hour_cos",
                F.cos(F.lit(2.0 * math.pi) * F.col("hour") / F.lit(24.0)))
    .withColumn("month_sin",
                F.sin(F.lit(2.0 * math.pi) * F.month("date") / F.lit(12.0)))
    .withColumn("month_cos",
                F.cos(F.lit(2.0 * math.pi) * F.month("date") / F.lit(12.0)))
    .withColumn("day_index",
                F.datediff("date", F.lit("2017-12-01").cast("date")).cast("double")))

bike_train = bike.filter("date < DATE '2018-08-01'").cache()
bike_valid = bike.filter(
    "date >= DATE '2018-08-01' AND date < DATE '2018-10-01'"
).cache()
bike_test = bike.filter("date >= DATE '2018-10-01'").cache()

rows = []
for name, df in [("train", bike_train), ("valid", bike_valid), ("test", bike_test)]:
    r = df.agg(F.count("*").alias("n"), F.min("date").alias("from"), F.max("date").alias("to"),
               F.avg("label").alias("mean_target")).first()
    rows.append({"Часть": name, "С": r["from"], "По": r["to"], "Строк": r["n"], "Средний спрос":
round(r["mean_target"], 1)})
print("Таблица 7. Временное разбиение Seoul Bike после исключения закрытых часов")
print(pd.DataFrame(rows).to_string(index=False))
assert sum(r["Строк"] for r in rows) == bike.count(), "Потеряны строки при разбиении"
print("Рабочих часов всего:", bike.count())
```

```
Таблица 7. Временное разбиение Seoul Bike после исключения закрытых часов
Часть          С          По  Строк  Средний спрос
train 2017-12-01 2018-07-31   5784          645.0
valid 2018-08-01 2018-09-29   1368          968.9
 test 2018-10-01 2018-11-30   1313          850.3
Рабочих часов всего: 8465
```

Разбиение совпадает с таблицей 7 методички: train 01.12.2017–31.07.2018 (5 784 ч), valid 01.08–30.09.2018 (1 368 ч), test 01.10–30.11.2018 (1 313 ч); в сумме 8 465 рабочих часов, потерь нет. Средний спрос заметно различается: 645 аренд/ч в train (зима, весна, начало лета), 969 в valid, 850 в test — поэтому часть разрыва метрик будет связана с изменением уровня спроса между сезонами, а не только с переобучением.

Практический блок 5.4. Pipeline регрессоров

Линейная ветка: индикаторы категорий + погодные и циклические координаты → `StandardScaler` → `LinearRegression` (L2). Полиномиальная ветка: `PolynomialExpansion(degree=2)` для пяти координат (температура, влажность, радиация, hour_sin, hour_cos — 20 слагаемых) и более сильная регуляризация. Лес: исходный числовой набор + `day_index`, без масштабирования.

```
BIKE_CAT = ["season", "holiday", "weekday"]
BIKE_NUM = [
    "temperature", "humidity", "wind_speed", "visibility",
    "dew_point", "solar_radiation", "rainfall", "snowfall",
    "hour_sin", "hour_cos", "month_sin", "month_cos"
]
BIKE_RF_NUM = BIKE_NUM + ["day_index"]
POLY_SOURCE = ["temperature", "humidity", "solar_radiation", "hour_sin", "hour_cos"]
POLY_REST = [c for c in BIKE_NUM if c not in POLY_SOURCE]


def bike_pipeline(kind):
    index_cols = [c + "_idx" for c in BIKE_CAT]
    encoded_cols = [c + "_ohe" for c in BIKE_CAT]
    indexer = StringIndexer(
        inputCols=BIKE_CAT, outputCols=index_cols,
        handleInvalid="keep", stringOrderType="alphabetAsc"
    )
    encoder = OneHotEncoder(
        inputCols=index_cols, outputCols=encoded_cols,
        handleInvalid="keep", dropLast=False
    )
    stages = [indexer, encoder]

    if kind == "LINEAR":
        stages += [
            VectorAssembler(inputCols=BIKE_NUM + encoded_cols, outputCol="features_raw"),
            StandardScaler(inputCol="features_raw", outputCol="features",
                           withMean=False, withStd=True),
            LinearRegression(maxIter=150, regParam=0.1, elasticNetParam=0.0,
                             standardization=False)
        ]
    elif kind == "POLY":
        stages += [
            VectorAssembler(inputCols=POLY_SOURCE, outputCol="poly_input"),
            PolynomialExpansion(degree=2, inputCol="poly_input", outputCol="poly_features"),
            VectorAssembler(inputCols=["poly_features"] + POLY_REST + encoded_cols,
                            outputCol="features_raw"),
            StandardScaler(inputCol="features_raw", outputCol="features",
                           withMean=False, withStd=True),
            LinearRegression(maxIter=200, regParam=1.0, elasticNetParam=0.0,
                             standardization=False)
        ]
    elif kind == "RF":
        stages += [
            VectorAssembler(inputCols=BIKE_RF_NUM + encoded_cols, outputCol="features"),
            RandomForestRegressor(
                numTrees=120, maxDepth=12, maxBins=64,
                minInstancesPerNode=5, featureSubsetStrategy="0.7",
                seed=SEED
            )
        ]
    else:
        raise ValueError(kind)
    return Pipeline(stages=stages)
```

5.5. Метрики и базовые правила

`regression_metrics` вызывает `RegressionEvaluator` для RMSE, MAE, R^2 и считает долю отрицательных прогнозов. Базовые правила: среднее train и среднее по часу суток (тоже только по train).

```
def regression_metrics(pred, label="label", prediction="prediction"):
    result = {}
```

```
    for metric in ["rmse", "mae", "r2"]:
        result[metric] = RegressionEvaluator(
            labelCol=label, predictionCol=prediction, metricName=metric
        ).evaluate(pred)
    result["negative_share"] = pred.agg(
        F.avg((F.col(prediction) < 0).cast("double"))
    ).first()[0]
    return result

train_mean = bike_train.agg(F.avg("label")).first()[0]
hour_means = bike_train.groupBy("hour").agg(
    F.avg("label").alias("hour_mean")
)

def bike_baseline(df, by_hour=False):
    if not by_hour:
        return df.withColumn("prediction", F.lit(train_mean))
    return (df.join(hour_means, "hour", "left")
        .withColumn("prediction", F.coalesce("hour_mean", F.lit(train_mean))))

print("Среднее train:", round(train_mean, 1))
```

```
Среднее train: 645.0
```

5.6. Обучение и сравнение

Три Pipeline обучаются на `bike_train`; лучшая обучаемая модель выбирается по минимальной RMSE valid и только затем применяется к test. После фиксации выбора для наглядности (рис. 7) на test считаются и остальные решения — на выбор это уже не влияет.

```
bike_models = {}
bike_rows = []
bike_valid_predictions = {}

for name, kind in [("Линейная регрессия", "LINEAR"),
                   ("Полиномиальная регрессия", "POLY"),
                   ("Случайный лес", "RF")]:
    sc.setJobGroup("P5_REG_" + kind, "Регрессия: " + name)
    t0 = perf_counter()
    model = bike_pipeline(kind).fit(bike_train)
    fit_s = perf_counter() - t0
    pred = model.transform(bike_valid).cache()
    pred.select(F.avg("prediction")).first()
    bike_models[name] = model
    bike_valid_predictions[name] = pred
    bike_rows.append({"model": name, "fit_s": fit_s, **regression_metrics(pred)})

for name, by_hour in [("Среднее train", False), ("Среднее по часу", True)]:
    bike_rows.append({"model": name, **regression_metrics(bike_baseline(bike_valid, by_hour))})

bike_validation = pd.DataFrame(bike_rows).sort_values("rmse")
print("Метрики на valid:")
print(bike_validation.round(3).to_string(index=False))

learned = bike_validation[bike_validation["model"].isin(bike_models)]
BEST_BIKE_NAME = str(learned.iloc[0]["model"])
BEST_BIKE = bike_models[BEST_BIKE_NAME]
print("\nВыбрана модель:", BEST_BIKE_NAME)

sc.setJobGroup("P5_REG_TEST", "Регрессия: однократная проверка на test")
bike_test_prediction = (BEST_BIKE.transform(bike_test)
    .select("date", "hour", "label", "prediction")
    .withColumn("residual", F.col("label") - F.col("prediction"))
    .cache())
```

```
bike_test_metrics = regression_metrics(bike_test_prediction)
print("TEST:", {k: round(v, 3) for k, v in bike_test_metrics.items()})
```

```
Метрики на valid:
                   model  fit_s    rmse     mae     r2  negative_share
           Случайный лес 56.863 285.735 196.853  0.801           0.000
Полиномиальная регрессия  1.820 401.083 296.099  0.607           0.002
      Линейная регрессия  2.792 525.596 421.225  0.326           0.004
         Среднее по часу    NaN 559.135 429.727  0.237           0.000
           Среднее train    NaN 717.361 539.660 -0.256           0.000

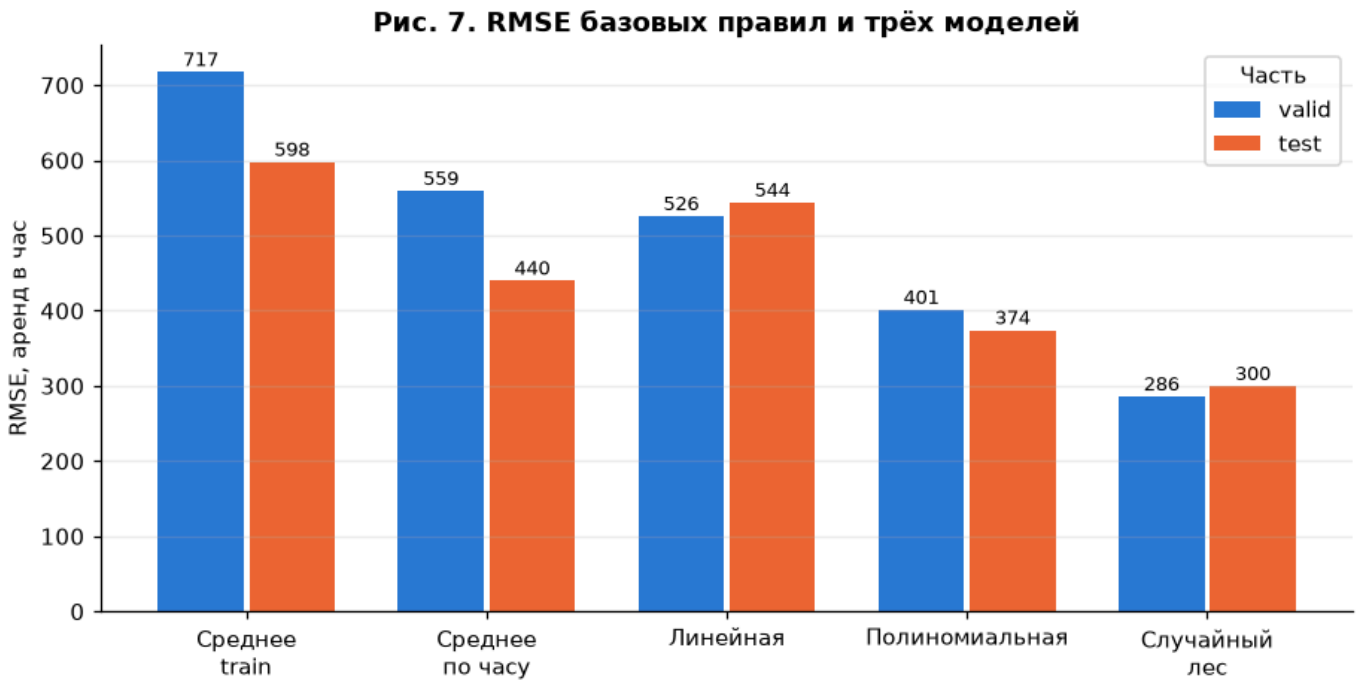
Выбрана модель: Случайный лес
TEST: {'rmse': 299.66, 'mae': 196.99, 'r2': 0.715, 'negative_share': 0.0}
```

```
# Таблица 8 и рис. 7: справочный пересчёт на test всех решений после фиксации выбора
test_rows = {}
for name, model in bike_models.items():
    test_rows[name] = regression_metrics(model.transform(bike_test))
for name, by_hour in [("Среднее train", False), ("Среднее по часу", True)]:
    test_rows[name] = regression_metrics(bike_baseline(bike_test, by_hour))

order = ["Среднее train", "Среднее по часу", "Линейная регрессия", "Полиномиальная регрессия", "Случайный
лес"]
v = bike_validation.set_index("model")
table8 = pd.DataFrame({
    "RMSE valid": [v.loc[m, "rmse"] for m in order],
    "MAE valid": [v.loc[m, "mae"] for m in order],
    "R² valid": [v.loc[m, "r2"] for m in order],
    "RMSE test": [test_rows[m]["rmse"] for m in order],
    "MAE test": [test_rows[m]["mae"] for m in order],
    "R² test": [test_rows[m]["r2"] for m in order],
    "Доля <0 test": [test_rows[m]["negative_share"] for m in order],
}, index=order)
print("Таблица 8. Метрики регрессии")
print(table8.round(3).to_string())

fig, ax = plt.subplots(figsize=(8.5, 4.4))
x = np.arange(len(order)); w = 0.38
for i, (col, color) in enumerate([("RMSE valid", C1), ("RMSE test", C2)]):
    bars = ax.bar(x + (i - 0.5) * w, table8[col], w * 0.94, color=color, label=col.replace("RMSE ", ""))
    for b, val in zip(bars, table8[col]):
        ax.text(b.get_x() + b.get_width() / 2, val + 8, f"{val:.0f}", ha="center", fontsize=8.5)
ax.set_xticks(x, ["Среднее\ntrain", "Среднее\nпо часу", "Линейная", "Полиномиальная", "Случайный\nлес"])
ax.set(ylabel="RMSE, аренд в час", title="Рис. 7. RMSE базовых правил и трёх моделей")
ax.legend(title="Часть"); ax.grid(axis="x", visible=False)
plt.tight_layout(); plt.savefig(OUT / "regression_rmse.png", dpi=160); plt.show()
```

```
Таблица 8. Метрики регрессии
                        RMSE valid  MAE valid  R² valid  RMSE test  MAE test  R² test  Доля <0 test
Среднее train              717.361    539.660    -0.256    597.657   463.526   -0.134         0.000
Среднее по часу            559.135    429.727     0.237    439.820   323.649    0.386         0.000
Линейная регрессия         525.596    421.225     0.326    543.784   404.041    0.061         0.076
Полиномиальная регрессия   401.083    296.099     0.607    374.058   251.286    0.556         0.015
Случайный лес              285.735    196.853     0.801    299.660   196.990    0.715         0.000
```



Сравнение на valid и test. Глобальное среднее train даёт $R^2 < 0$ и на valid (−0,256), и на test (−0,134): средний спрос в этих периодах выше, чем в train, и константа хуже среднего самой оцениваемой части. Среднее по часу (RMSE 559 / 440) сразу заметно сильнее — суточный ритм объясняет большую часть спроса. Линейная регрессия на valid лишь немного лучше этого правила (526), а на test даже хуже (544, R^2 = 0,06). Полиномиальное расширение снижает RMSE до 401 / 374: квадраты и взаимодействия температуры, влажности, радиации и часа действительно описывают нелинейность. Случайный лес — лучший на valid (RMSE 286, MAE 197, R^2 0,80) и выбран; на test он сохраняет преимущество: RMSE 300, MAE 197, R^2 0,715 (контроль методички: 288 / 317). Небольшой рост RMSE на test согласуется со сдвигом сезона. Обучение леса (≈51 с) на порядок дольше линейных моделей (≈1,6 с), но выигрыш в ошибке это оправдывает.

5.7. Проверка итоговой модели

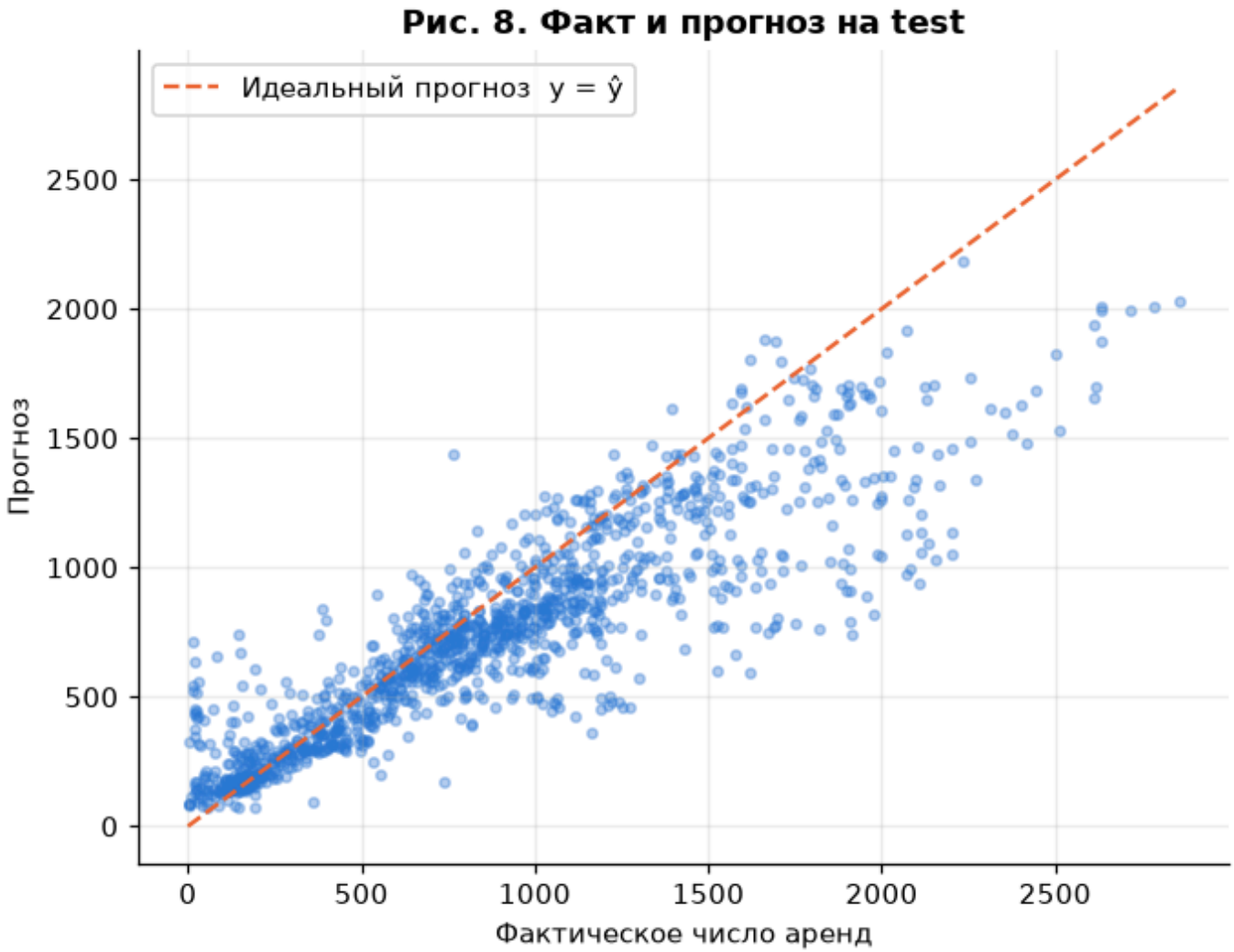
В pandas передаётся только тестовая таблица (≈1,3 тыс. строк: дата, час, факт, прогноз).

```
reg_local = bike_test_prediction.orderBy("date", "hour").toPandas()
reg_local["timestamp"] = (
    pd.to_datetime(reg_local["date"]) +
    pd.to_timedelta(reg_local["hour"], unit="h")
)

fig, ax = plt.subplots(figsize=(6.4, 5.0))
ax.scatter(reg_local["label"], reg_local["prediction"], s=12, alpha=0.35, color=C1)
limit = max(reg_local["label"].max(), reg_local["prediction"].max())
ax.plot([0, limit], [0, limit], "--", color=C2, label="Идеальный прогноз  y = ŷ")
ax.set(xlabel="Фактическое число аренд", ylabel="Прогноз",
       title="Рис. 8. Факт и прогноз на test")
ax.legend()
plt.tight_layout(); plt.savefig(OUT / "regression_scatter.png", dpi=160); plt.show()

week = reg_local[reg_local["timestamp"] < reg_local["timestamp"].min() + pd.Timedelta(days=7)]
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(week["timestamp"], week["label"], label="Факт", color=C1, linewidth=1.8)
ax.plot(week["timestamp"], week["prediction"], label="Прогноз", color=C2, linewidth=1.8)
ax.set(xlabel="Дата и час", ylabel="Число аренд", title="Рис. 9. Первая неделя тестового периода")
ax.legend()
plt.tight_layout(); plt.savefig(OUT / "regression_week.png", dpi=160); plt.show()
```

```
mae_hour = (bike_test_prediction.groupBy("hour")
    .agg(F.avg(F.abs("residual")).alias("mae"), F.avg("label").alias("mean_label"))
    .orderBy("hour").toPandas())
fig, ax = plt.subplots(figsize=(9, 4))
ax.bar(mae_hour["hour"], mae_hour["mae"], color=C1, width=0.7)
ax.set_xticks(range(0, 24))
ax.set(xlabel="Час суток", ylabel="MAE, аренд", title="Рис. 10. Средняя абсолютная ошибка по часу суток
(test)")
ax.grid(axis="x", visible=False)
plt.tight_layout(); plt.savefig(OUT / "regression_mae_by_hour.png", dpi=160); plt.show()
print(mae_hour.sort_values("mae", ascending=False).head(5).round(1).to_string(index=False))
```







hour	mae	mean_label
8	672.7	1356.2
18	525.1	1824.6
7	387.9	765.9
19	256.3	1335.5
9	253.2	832.2

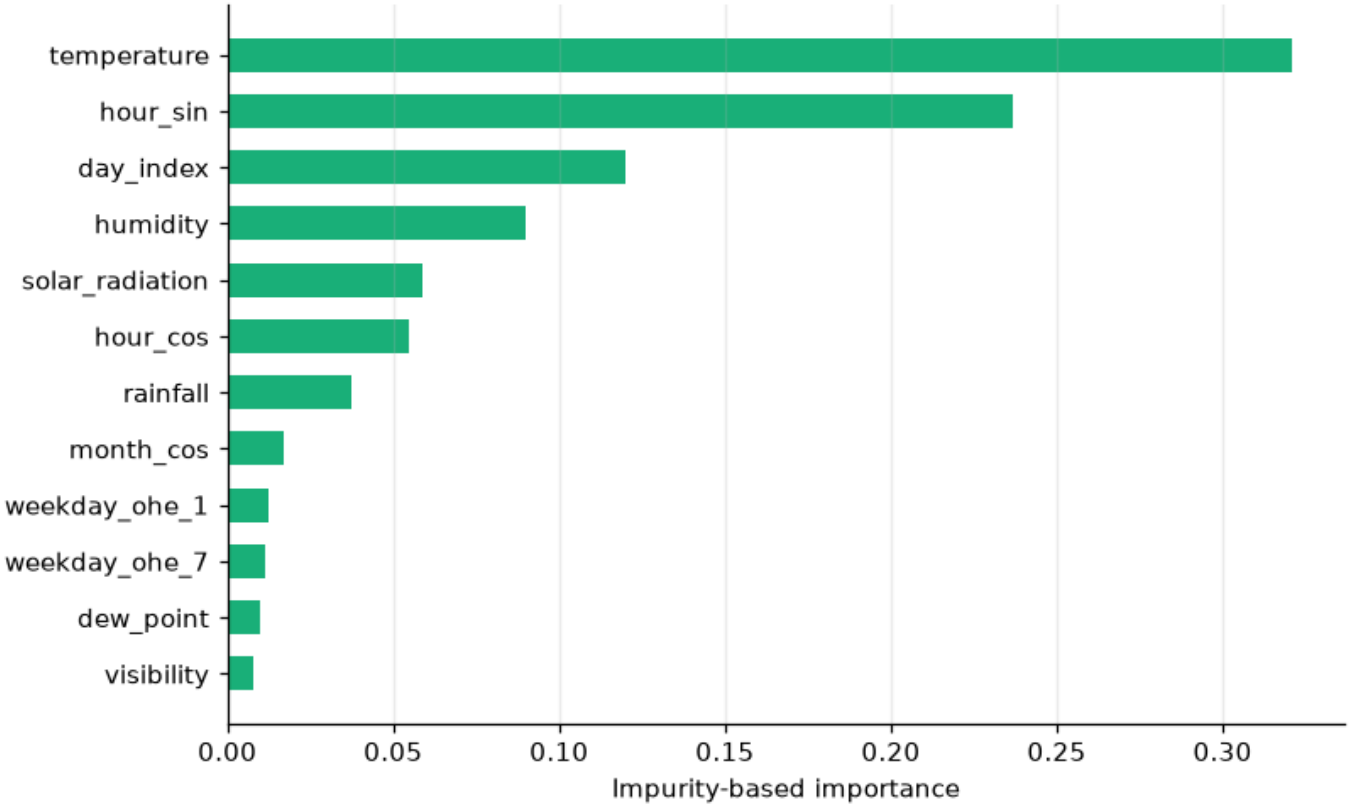
Анализ ошибок. На диаграмме рассеяния разброс растёт вместе со спросом: малые и средние значения восстанавливаются точно, а крупные пики (2 000–3 500) лес систематически занижает — он усредняет листья и не выходит за пределы наблюдённых комбинаций (гетероскедастичность ошибки). Недельный ряд показывает, что модель воспроизводит суточную форму (утренний и вечерний пики, ночной спад), а основная ошибка — в высоте пиков. Профиль MAE по часу это подтверждает: максимальная ошибка в 8:00 (673) и 18:00 (525) — часы резкой смены спроса; ночью ошибка мала. Улучшение возможно за счёт признаков «будний день × час пика» или отдельной модели для пиковых часов.

5.8. Важность признаков и ограничения

```
rf_bike = bike_models["Случайный лес"]
bike_names = vector_names(rf_bike, bike_valid, "features")
bike_imp = (pd.Series(rf_bike.stages[-1].featureImportances.toArray(), index=bike_names)
            .sort_values(ascending=False))
top = bike_imp.head(12)[::-1]
fig, ax = plt.subplots(figsize=(7.5, 4.8))
ax.barh(top.index, top.values, color=C3, height=0.6)
ax.set(xlabel="Impurity-based importance", title="Рис. 11. Наиболее используемые признаки лесной
регрессии")
ax.grid(axis="y", visible=False)
plt.tight_layout(); plt.savefig(OUT / "regression_importance.png", dpi=160); plt.show()
print(bike_imp.head(8).round(4).to_string())

# Отрицательные прогнозы линейных моделей и правило max(0, prediction), выбираемое по valid
print("\nОтрицательные прогнозы и обрезка max(0, ŷ):")
for name in ["Линейная регрессия", "Полиномиальная регрессия"]:
    base = bike_valid_predictions[name]
    clipped = base.withColumn("prediction_clip", F.greatest(F.col("prediction"), F.lit(0.0)))
    m0 = regression_metrics(base)
    m1 = regression_metrics(clipped, prediction="prediction_clip")
    print(f"  {name:26s} доля <0: valid {m0['negative_share']:.3f}, test {test_rows[name]
['negative_share']:.3f} | "
          f"RMSE valid {m0['rmse']:.1f} -> с обрезкой {m1['rmse']:.1f}; MAE {m0['mae']:.1f} ->
{m1['mae']:.1f}")
```

Рис. 11. Наиболее используемые признаки лесной регрессии




```
temperature        0.3207
hour_sin           0.2370
day_index          0.1201
humidity           0.0898
solar_radiation    0.0587
hour_cos           0.0545
rainfall           0.0373
month_cos          0.0169

Отрицательные прогнозы и обрезка max(0, ŷ):
  Линейная регрессия         доля <0: valid 0.004, test 0.076 | RMSE valid 525.6 -> с обрезкой 524.5; MAE
421.2 -> 420.0
  Полиномиальная регрессия   доля <0: valid 0.002, test 0.015 | RMSE valid 401.1 -> с обрезкой 400.9; MAE
296.1 -> 295.8
```

Важность и ограничения. Лес чаще всего использует температуру (32 %), `hour_sin` (24 %) и `day_index` (12 %), затем влажность и солнечную радиацию. Высокая важность `day_index` означает опору на положение во времени — внутри одного года это помогает, но перенос на другой год требует новой проверки. Линейные модели дают отрицательные прогнозы числа аренд: на test 7,6 % у линейной и 1,5 % у полиномиальной (в методичке 7,6 % и 1,2 %). Правило `max(0, ŷ)` проверено на valid: RMSE линейной модели 525,6 → 524,5, полиномиальной 401,1 → 400,9 — эффект мал, потому что на valid (лето) отрицательных прогнозов почти нет (0,4 % и 0,2 %); проблема проявляется в осеннем test, но вывод о выборе модели обрезка не меняет.

4. Кластеризация интернет-сессий

6.1–6.2. Признаки и подготовка вектора

Вектор: 6 логарифмированных счётчиков и длительностей + `BounceRates`, `ExitRates`; все 8 координат центрируются и масштабируются (`withMean=True` допустимо: вектор плотный и маленький). `Revenue` и `PageValues` в расстояние не входят; `Revenue` подключается только после выбора k для описания групп.

```
CLUSTER_LOG_SOURCE = [
    "Administrative", "Administrative_Duration",
    "Informational", "Informational_Duration",
    "ProductRelated", "ProductRelated_Duration"
]
CLUSTER_LOG = ["cluster_log_" + c for c in CLUSTER_LOG_SOURCE]
CLUSTER_RATE = ["BounceRates", "ExitRates"]

cluster_expressions = [
    f"log1p(`{source}`) AS `{target}`"
    for source, target in zip(CLUSTER_LOG_SOURCE, CLUSTER_LOG)
]
cluster_prep = Pipeline(stages=[
    SQLTransformer(
        statement="SELECT *, " + ", ".join(cluster_expressions) + " FROM __THIS__"
    ),
    VectorAssembler(inputCols=CLUSTER_LOG + CLUSTER_RATE, outputCol="cluster_raw"),
    StandardScaler(inputCol="cluster_raw", outputCol="features",
                   withMean=True, withStd=True)
])

cluster_prep_model = cluster_prep.fit(shop_raw)
shop_vectors = cluster_prep_model.transform(shop_raw).select("row_id", "features").cache()

sample_order = F.sha2(F.concat_ws(
    ":", F.col("row_id").cast("string"), F.lit(str(SEED))
), 256)
cluster_sample = shop_vectors.orderBy(sample_order, "row_id").limit(4000).cache()
print("Векторов:", shop_vectors.count(), "| выборка для силуэта:", cluster_sample.count())
```

```
Векторов: 12330 | выборка для силуэта: 4000
```

6.3. KMeans и выбор k

KMeans обучается для k = 2…8 на полном распределённом наборе. Silhouette считается scikit-learn на одной и той же детерминированной выборке 4 000 строк (евклидово расстояние); дополнительно для сверки приведён силуэт Spark `ClusteringEvaluator` на всём наборе (он использует квадрат евклидова расстояния, поэтому значения выше по шкале, но сравнимы между k).

```
k_models = {}
k_rows = []
spark_silhouette = ClusteringEvaluator(featuresCol="features", predictionCol="prediction",
                                       metricName="silhouette", distanceMeasure="squaredEuclidean")

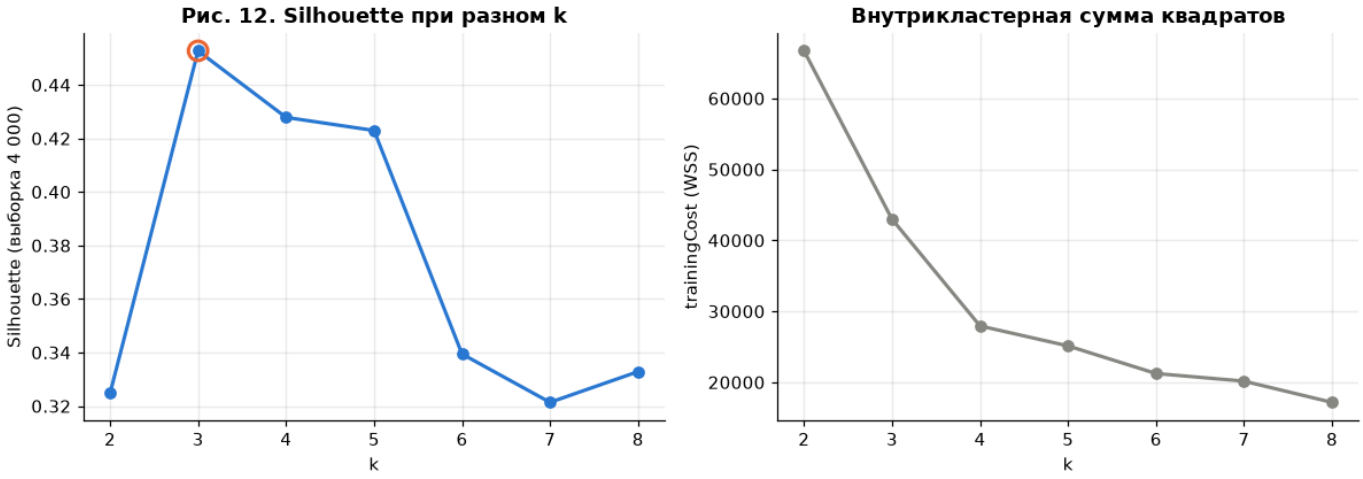
for k in range(2, 9):
    sc.setJobGroup("P5_CLUSTER_" + str(k), f"KMeans k={k}")
    model = KMeans(k=k, seed=SEED, maxIter=60, tol=1e-4).fit(shop_vectors)
    sample_pred = model.transform(cluster_sample).select("features", "prediction")
    local = sample_pred.toPandas()
    x = np.vstack([v.toArray() for v in local["features"]])
    labels = local["prediction"].to_numpy()
    silhouette = silhouette_score(x, labels, metric="euclidean")
```

```
    full_pred = model.transform(shop_vectors)
    sizes = full_pred.groupBy("prediction").count()
    k_rows.append({
        "k": k,
        "silhouette": silhouette,
        "silhouette_spark_sq": spark_silhouette.evaluate(full_pred),
        "training_cost": model.summary.trainingCost,
        "min_cluster": sizes.agg(F.min("count")).first()[0]
    })
    k_models[k] = model

k_table = pd.DataFrame(k_rows).sort_values(["silhouette", "k"], ascending=[False, True])
print("Таблица 10. Подбор числа кластеров")
print(k_table.round(4).to_string(index=False))
BEST_K = int(k_table.iloc[0]["k"])
BEST_KMEANS = k_models[BEST_K]
print("Выбрано k =", BEST_K)

kt = k_table.sort_values("k")
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].plot(kt["k"], kt["silhouette"], marker="o", color=C1, linewidth=2)
axes[0].scatter([BEST_K], [kt.set_index("k").loc[BEST_K, "silhouette"]], s=120,
                facecolors="none", edgecolors=C2, linewidths=2, zorder=3)
axes[0].set(xlabel="k", ylabel="Silhouette (выборка 4 000)", title="Рис. 12. Silhouette при разном k")
axes[1].plot(kt["k"], kt["training_cost"], marker="o", color=GRAY, linewidth=2)
axes[1].set(xlabel="k", ylabel="trainingCost (WSS)", title="Внутрикластерная сумма квадратов")
plt.tight_layout(); plt.savefig(OUT / "clusters_silhouette.png", dpi=160); plt.show()
```

```
Таблица 10. Подбор числа кластеров
 k  silhouette  silhouette_spark_sq  training_cost  min_cluster
 3      0.4527               0.6380     42977.0833          993
 4      0.4278               0.5989     27939.9723          880
 5      0.4230               0.5951     25125.5619          811
 6      0.3394               0.4903     21233.3343          781
 8      0.3329               0.4903     17170.2615          509
 2      0.3248               0.4567     66703.7521         6136
 7      0.3214               0.4563     20159.2825          781
Выбрано k = 3
```



Выбор k. Максимум силуэта — при k = 3 (0,453 на выборке 4 000; в методичке 0,444), k = 4 и 5 близки (0,428 и 0,423), далее силуэт падает. Силуэт Spark (квадрат евклидова расстояния, весь набор) даёт тот же порядок: максимум 0,638 при k = 3. `trainingCost` монотонно убывает с 66 704 до 17 170 и минимума не имеет — критерием быть не может. Минимальный кластер при k = 3 содержит 993 сессии, т.е. высокий силуэт не получен ценой почти пустой группы. Выбрано k = 3.

6.4. Профили и двумерная проекция

Метки соединяются с исходными строками по `row_id`; профиль строится в исходных единицах. PCA обучается только для визуализации.

```
cluster_assignments = (BEST_KMEANS.transform(shop_vectors)
    .select("row_id", F.col("prediction").alias("cluster")))

cluster_profiles = (shop_raw.join(cluster_assignments, "row_id")
    .groupBy("cluster")
    .agg(
        F.count("*").alias("n"),
        F.avg("Administrative").alias("administrative"),
        F.avg("Informational").alias("informational"),
        F.avg("ProductRelated").alias("product_pages"),
        F.avg("ProductRelated_Duration").alias("product_duration"),
        F.avg("BounceRates").alias("bounce_rate"),
        F.avg("ExitRates").alias("exit_rate"),
        F.avg(F.col("Revenue").cast("double")).alias("purchase_share")
    )
    .orderBy("cluster"))
profiles_pd = cluster_profiles.toPandas()
print("Таблица 11. Профили кластеров")
print(profiles_pd.round(3).to_string(index=False))

pca_model = PCA(k=2, inputCol="features", outputCol="pca_features").fit(shop_vectors)
print("Доля объяснённой дисперсии PC1, PC2:", np.round(pca_model.explainedVariance.toArray(), 3))
pca_rows = (pca_model.transform(BEST_KMEANS.transform(shop_vectors))
    .orderBy(sample_order, "row_id")
    .limit(4000)
    .select("prediction", "pca_features")
    .toPandas())
coords = np.vstack([v.toArray() for v in pca_rows["pca_features"]])

fig, ax = plt.subplots(figsize=(7.2, 5.2))
for cluster in sorted(pca_rows["prediction"].unique()):
    mask = pca_rows["prediction"].to_numpy() == cluster
    ax.scatter(coords[mask, 0], coords[mask, 1], s=9, alpha=0.4,
               color=PALETTE[int(cluster)], label=f"Кластер {cluster}")
ax.set(xlabel="Первая главная компонента", ylabel="Вторая главная компонента",
       title="Рис. 13. Проекция сессий на две главные компоненты")
ax.legend(markerscale=2)
plt.tight_layout(); plt.savefig(OUT / "clusters_pca.png", dpi=160); plt.show()

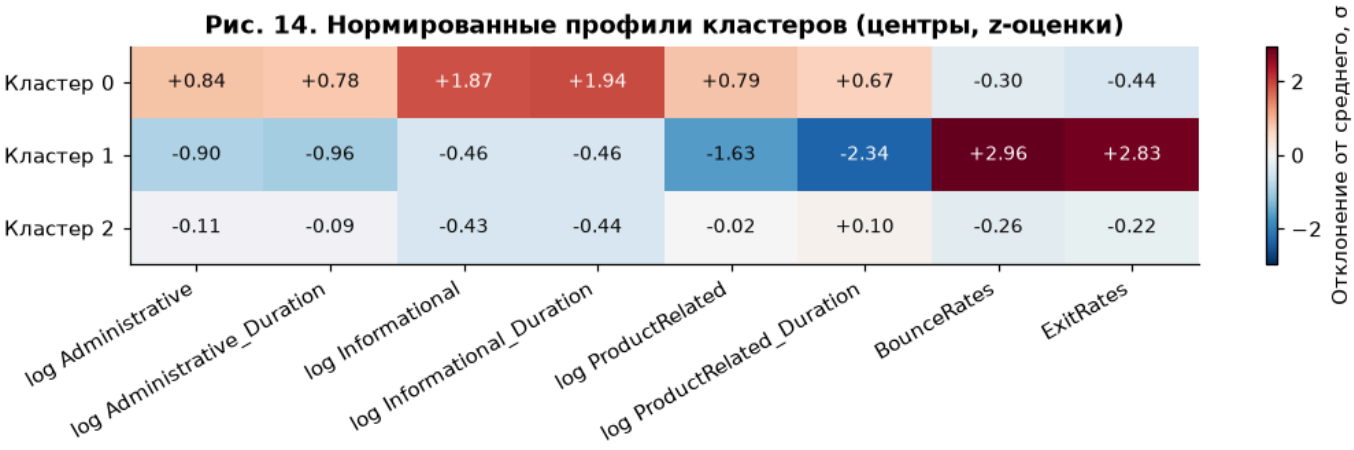
# Рис. 14. Нормированные профили: центры KMeans в стандартизованном пространстве
centers = pd.DataFrame(np.vstack(BEST_KMEANS.clusterCenters()),
                       columns=[c.replace("cluster_log_", "log ") for c in CLUSTER_LOG] + CLUSTER_RATE)
lim = np.abs(centers.values).max()
fig, ax = plt.subplots(figsize=(10, 0.9 + 0.75 * BEST_K))
im = ax.imshow(centers.values, cmap="RdBu_r", vmin=-lim, vmax=lim, aspect="auto")
for i in range(centers.shape[0]):
    for j in range(centers.shape[1]):
        val = centers.values[i, j]
        ax.text(j, i, f"{val:+.2f}", ha="center", va="center", fontsize=9,
                color="white" if abs(val) > lim * 0.6 else "#0b0b0b")
ax.set_xticks(range(centers.shape[1]), centers.columns, rotation=30, ha="right")
ax.set_yticks(range(centers.shape[0]), [f"Кластер {i}" for i in range(centers.shape[0])])
ax.grid(False)
ax.set_title("Рис. 14. Нормированные профили кластеров (центры, z-оценки)")
fig.colorbar(im, ax=ax, label="Отклонение от среднего, σ")
plt.tight_layout(); plt.savefig(OUT / "clusters_profiles.png", dpi=160); plt.show()
```

Таблица 11. Профили кластеров

cluster	n	administrative	informational	product_pages	product_duration	bounce_rate	exit_rate	purchase_share
0	2305	5.130	2.537	65.677	2599.674	0.007	0.022	0.239
1	993	0.030	0.017	2.285	32.816	0.170	0.184	0.004
2	9032	1.848	0.038	26.306	963.949	0.010	0.033	0.150

Доля объяснённой дисперсии PC1, PC2: [0.531 0.216]





Интерпретация кластеров (номера технические):

- **Кластер 1 — «отказы», 993 сессии (8 %).** ≈2 товарные страницы и 33 с на товарах, очень высокие BounceRates 0,170 и ExitRates 0,184 (на тепловой карте +2,96σ и +2,83σ). Покупкой заканчиваются 0,4 % сессий.

- **Кластер 0 — «активные исследователи», 2 305 сессий (19 %).** ≈66 товарных страниц, ≈2 600 с на товарах, больше административных и особенно информационных просмотров (+1,9σ), низкие отказы. Доля покупок наибольшая — 23,9 %.
- **Кластер 2 — «обычные сессии», 9 032 (73 %).** Средние значения почти всех признаков (≈26 страниц, ≈960 с), информационные страницы почти не открываются; доля покупок 15,0 % — около общей.

Revenue в построении не участвовал, но доли покупок между группами различаются в 60 раз — найденная поведенческая структура связана с результатом. PCA сохраняет 75 % дисперсии (53 % + 22 %); на проекции кластер «отказов» компактно отделён, а два других соприкасаются — граница между ними задаётся в основном уровнем активности и не является «пустым промежутком». Проекция иллюстрирует геометрию, но не доказывает разделение в 8-мерном пространстве.

5. Сохранение и повторное применение моделей

Сохраняются полные `PipelineModel` (предобработка + модель), а порог классификации и параметры применения — в отдельном JSON-паспорте. Дополнительно модели загружаются обратно и проверяется, что они воспроизводят те же предсказания.

```
BEST_SHOP.write().overwrite().save((OUT / "shop_classifier").as_posix())
BEST_BIKE.write().overwrite().save((OUT / "bike_regressor").as_posix())

cluster_full_pipeline = PipelineModel(
    stages=cluster_prep_model.stages + [BEST_KMEANS]
)
cluster_full_pipeline.write().overwrite().save((OUT / "shop_kmeans").as_posix())

config = {
    "seed": SEED,
    "classification_model": BEST_SHOP_NAME,
    "classification_threshold": SHOP_THRESHOLD,
    "classification_features_note": "PageValues включён: прогноз по накопленному состоянию сессии",
    "regression_model": BEST_BIKE_NAME,
    "regression_scope": "только часы Functioning Day = Yes; погода — фактическая за час",
    "kmeans_k": BEST_K,
    "spark_version": spark.version
}
(OUT / "model_config.json").write_text(
    json.dumps(config, ensure_ascii=False, indent=2), encoding="utf-8"
)
print(json.dumps(config, ensure_ascii=False, indent=2))

# Проверка повторного применения.
# PipelineModel.load() в PySpark 3.5 читает metadata через Python RDD (sc.textFile(...).first()),
# а на этом стенде (Windows, Python 3.12) Python-воркер Spark аварийно завершается.
# Все стадии сохранённых цепочек — JVM-объекты, поэтому применяем Java-ридер: результат тот же
PipelineModel.
from pyspark.ml.util import JavaMLReader

def load_pipeline(path):
    return JavaMLReader(PipelineModel).load(Path(path).as_posix())

cfg = json.loads((OUT / "model_config.json").read_text(encoding="utf-8"))
shop_loaded = load_pipeline(OUT / "shop_classifier")
bike_loaded = load_pipeline(OUT / "bike_regressor")
kmeans_loaded = load_pipeline(OUT / "shop_kmeans")

reloaded = binary_scores(shop_loaded, shop_test).withColumnRenamed("score", "score_loaded")
diff = shop_test_scores.join(reloaded, ["row_id", "label"]).agg(F.max(F.abs(F.col("score") -
F.col("score_loaded")))).first()[0]
print("\nМакс. расхождение score классификатора после load:", diff)
print("Метрики test с порогом из JSON совпадают:",
      binary_metrics(binary_scores(shop_loaded, shop_test), cfg["classification_threshold"], ranking=False)
["f1"]
      == shop_test_metrics["f1"])
print("RMSE test регрессора после load:", round(regression_metrics(bike_loaded.transform(bike_test))
["rmse"], 3))
same_clusters = (kmeans_loaded.transform(shop_raw).select("row_id", "prediction")
                 .join(cluster_assignments, "row_id")
                 .filter(F.col("prediction") != F.col("cluster")).count())
print("Строк с другим кластером после load:", same_clusters)
```

```
{
  "seed": 42,
  "classification_model": "Случайный лес",
  "classification_threshold": 0.538382750703671,
  "classification_features_note": "PageValues включён: прогноз по накопленному состоянию сессии",
  "regression_model": "Случайный лес",
  "regression_scope": "только часы Functioning Day = Yes; погода — фактическая за час",
  "kmeans_k": 3,
  "spark_version": "3.5.3"
}
Макс. расхождение score классификатора после load: 0.0
Метрики test с порогом из JSON совпадают: True
RMSE test регрессора после load: 299.66
Строк с другим кластером после load: 0
```

Практический блок 7.1. Освобождение кэша

```
for pred in list(shop_valid_predictions.values()) + list(bike_valid_predictions.values()):
    pred.unpersist()
for df in [shop_split, shop_test_scores, without_scores, bike_train, bike_valid,
           bike_test, bike_test_prediction, shop_vectors, cluster_sample]:
    df.unpersist()
sc.setJobGroup("P5_SELF", "Самостоятельная работа")
print("Основная часть готова. Результаты сохранены в", OUT)
print(sorted(p.name for p in OUT.iterdir()))
```

```
Основная часть готова. Результаты сохранены в C:\Users\relax\Documents\TiIABD\Практическая работа №5\practice5_results
['bike_regressor', 'classification_ablation.png', 'classification_classes.png', 'classification_confusion.png', 'classification_importance.png', 'classification_pr.png', 'classification_roc.png', 'classification_threshold.png', 'clusters_pca.png', 'clusters_profiles.png', 'clusters_silhouette.png', 'model_config.json', 'regression_importance.png', 'regression_mae_by_hour.png', 'regression_rmse.png', 'regression_scatter.png', 'regression_week.png', 'shop_classifier', 'shop_kmeans']
```

6. Самостоятельная работа: регрессия времени прослушивания подкастов

Набор данных: `Podcast Dataset.zip` → `podcast_dataset.csv` (52 500 строк × 11 столбцов), выдан вместе с практической работой; описание полей — в `Passport.txt` архива. Одна строка — эпизод подкаста с характеристиками публикации.

Задача — регрессия: по признакам, известным в момент публикации эпизода (подкаст, жанр, длительность, популярность ведущего и гостя, число рекламных вставок, день/время публикации, тональность), спрогнозировать среднее фактическое время прослушивания `Listening_Time_minutes`. Прикладной смысл — заранее оценить удержание аудитории и спланировать размещение рекламы.

Модели: линейная регрессия, случайный лес, градиентный бустинг (GBT) в Spark MLlib; сравнение с двумя базовыми правилами; гиперпараметры подбираются минимизацией RMSE на valid (для линейной модели дополнительно — `CrossValidator`).

8.1. Загрузка данных и Spark-схема

Архив распаковывается блокнотом в рабочий каталог, к строкам добавляется устойчивый `row_id` (номер строки исходного CSV). Пустые ячейки CSV становятся `null`. Имена столбцов заменяются на короткие безопасные имена по порядку.

```
POD_ZIP = ROOT / "Podcast Dataset.zip"
POD_DIR = WORK / "podcast"
with zipfile.ZipFile(POD_ZIP) as zf:
    print("Состав архива:", [(i.filename, i.file_size) for i in zf.infolist()])
    zf.extractall(POD_DIR)
POD_SOURCE = POD_DIR / "podcast_dataset.csv"
POD_WORK = WORK / "podcast_with_id.csv"

with POD_SOURCE.open("r", encoding="utf-8-sig", newline="") as src, \
     POD_WORK.open("w", encoding="utf-8", newline="") as dst:
    reader, writer = csv.reader(src), csv.writer(dst)
    pod_header = next(reader)
    writer.writerow(["row_id"] + pod_header)
    for row_id, row in enumerate(reader, start=1):
        writer.writerow([row_id] + row)

pod_schema = StructType([
    StructField("row_id", LongType(), False),
    StructField("podcast", StringType(), False),       # Podcast_Name
    StructField("episode", StringType(), False),       # Episode_Title
    StructField("length", DoubleType(), True),         # Episode_Length_minutes
    StructField("genre", StringType(), False),         # Genre
    StructField("host_pop", DoubleType(), False),      # Host_Popularity_percentage
    StructField("day", StringType(), False),           # Publication_Day
    StructField("time_of_day", StringType(), False),   # Publication_Time
    StructField("guest_pop", DoubleType(), True),      # Guest_Popularity_percentage
    StructField("ads", IntegerType(), False),          # Number_of_Ads
    StructField("sentiment", StringType(), False),     # Episode_Sentiment
    StructField("label", DoubleType(), True),          # Listening_Time_minutes (цель)
])
for original, safe in zip(["row_id"] + pod_header, pod_schema.fieldNames()):
    print(f"{original:30s} -> {safe}")

pod_raw = (spark.read
    .option("header", True)
```

```
    .option("mode", "FAILFAST")
    .schema(pod_schema)
    .csv(POD_WORK.resolve().as_posix())
    .cache())
print("\nСтрок:", pod_raw.count())
pod_raw.printSchema()
pod_raw.show(5, truncate=False)
```

```
Состав архива: [('podcast_dataset.csv', 4596919), ('Passport.txt', 1684)]
row_id                          -> row_id
Podcast_Name                    -> podcast
Episode_Title                   -> episode
Episode_Length_minutes          -> length
Genre                           -> genre
Host_Popularity_percentage      -> host_pop
Publication_Day                 -> day
Publication_Time                -> time_of_day
Guest_Popularity_percentage     -> guest_pop
Number_of_Ads                   -> ads
Episode_Sentiment               -> sentiment
Listening_Time_minutes          -> label
Строк: 52500
root
 |-- row_id: long (nullable = true)
 |-- podcast: string (nullable = true)
 |-- episode: string (nullable = true)
 |-- length: double (nullable = true)
 |-- genre: string (nullable = true)
 |-- host_pop: double (nullable = true)
 |-- day: string (nullable = true)
 |-- time_of_day: string (nullable = true)
 |-- guest_pop: double (nullable = true)
 |-- ads: integer (nullable = true)
 |-- sentiment: string (nullable = true)
 |-- label: double (nullable = true)

+------+---------------+----------+------+----------+--------+--------+-----------+---------+---+---------+-
----------+
|row_id|podcast        |episode   |length|genre     |host_pop|day
|time_of_day|guest_pop|ads|sentiment|label      |
+------+---------------+----------+------+----------+--------+--------+-----------+---------+---+---------+-
----------+
|1     |Healthy Living |Episode 77|99.25 |Health    |21.37   |Thursday|Night      |70.22    |3  |Positive
|55.15869493|
|2     |Mystery Matters|Episode 6 |19.43 |True Crime|47.19   |Friday  |Evening    |75.15    |1  |Neutral
|7.686558813|
|3     |Current Affairs|Episode 1 |117.03|News      |96.33   |Sunday  |Night      |57.95    |3  |Neutral
|110.064645 |
|4     |Mystery Matters|Episode 38|16.97 |True Crime|25.73   |Monday  |Night      |24.19    |0  |Positive
|12.00038044|
|5     |Humor Hub      |Episode 73|83.48 |Comedy    |76.69   |Tuesday |Afternoon  |42.31    |3  |Positive
|NULL       |
+------+---------------+----------+------+----------+--------+--------+-----------+---------+---+---------+-
----------+
only showing top 5 rows
```

8.2. Распределение ключей объектов и схема разбиения

Перед разбиением нужно понять, что является «объектом»: строка, эпизод (пара подкаст + название) или подкаст целиком. Если один объект встречается в нескольких строках, случайное построчное разбиение отправит его копии и в train, и в test — это утечка.

```
n_rows = pod_raw.count()
key_stats = pod_raw.agg(
```

```
    F.countDistinct("podcast").alias("podcasts"),
    F.countDistinct("episode").alias("episode_titles"),
    F.countDistinct("podcast", "episode").alias("podcast_episode_pairs"),
).first().asDict()
print("Строк:", n_rows, "|", key_stats)

per_pair = pod_raw.groupBy("podcast", "episode").agg(
    F.count("*").alias("rows"),
    F.countDistinct("length").alias("distinct_length"),
    F.countDistinct("genre").alias("distinct_genre"),
    F.countDistinct("day").alias("distinct_day"))
per_pair.select(
    F.min("rows"), F.expr("percentile(rows, 0.5)").alias("median_rows"), F.max("rows"),
    F.avg("distinct_length").alias("avg_distinct_length"),
    F.max("distinct_genre").alias("max_genre_per_pair"),
    F.avg("distinct_day").alias("avg_distinct_day")).show()

per_podcast = pod_raw.groupBy("podcast").agg(F.count("*").alias("rows"),
                                             F.countDistinct("genre").alias("genres"))
per_podcast.select(F.min("rows"), F.max("rows"), F.max("genres").alias("max_genres_per_podcast")).show()

value_cols = [c for c in pod_raw.columns if c != "row_id"]
dup_groups = pod_raw.groupBy(*value_cols).count().filter("count > 1")
print("Полностью совпадающих строк (кроме row_id):",
      dup_groups.agg(F.sum("count")).first()[0], "в", dup_groups.count(), "группах;",
      "лишних копий:", n_rows - pod_raw.select(*value_cols).distinct().count())
print("Пропуски:", {k: v for k, v in null_counts(pod_raw).items() if v})

pp = per_pair.select("rows").toPandas()     # 4 799 строк — безопасно
fig, ax = plt.subplots(figsize=(7.5, 3.6))
ax.hist(pp["rows"], bins=range(1, pp["rows"].max() + 2), color=C1, edgecolor="white")
ax.set(xlabel="Строк на пару (подкаст, название эпизода)", ylabel="Число пар",
       title="Распределение числа строк на ключ эпизода")
ax.grid(axis="x", visible=False)
plt.tight_layout(); plt.savefig(OUT / "self_key_distribution.png", dpi=160); plt.show()
```

```
Строк: 52500 | {'podcasts': 48, 'episode_titles': 100, 'podcast_episode_pairs': 4799}
+---------+-----------+---------+------------------+-----------------+----------------+
|min(rows)|median_rows|max(rows)|avg_distinct_length|max_genre_per_pair|avg_distinct_day|
+---------+-----------+---------+------------------+-----------------+----------------+
|        1|       11.0|       25|  9.365909564492602|                1|5.40675140654303|
+---------+-----------+---------+------------------+-----------------+----------------+

+---------+---------+----------------------+
|min(rows)|max(rows)|max_genres_per_podcast|
+---------+---------+----------------------+
|      987|     1167|                     1|
+---------+---------+----------------------+

Полностью совпадающих строк (кроме row_id): 4933 в 2433 группах; лишних копий: 2500
Пропуски: {'length': 5246, 'guest_pop': 5258, 'label': 5395}
```



Вывод о ключах. 52 500 строк приходятся на 48 подкастов (987–1 167 строк на подкаст, жанр у подкаста один) и всего 100 названий «Episode N», образующих 4 799 пар (подкаст, эпизод) — в среднем 11 строк на пару (от 1 до 25). Внутри одной пары в среднем 9,4 разных длительности и 5,4 разных дня публикации, т.е. пара — не один и тот же эпизод, а скорее повторно используемая метка; но строки одной пары нельзя считать гарантированно независимыми. Кроме того, найдено 2 500 лишних полных копий строк (4 933 строки в 2 433 группах): при построчном разбиении копии попали бы одновременно в train и test. Пропуски: длительность 5 246 (10 %), популярность гостя 5 258 (10 %), цель 5 395 (10,3 %).

Схема разбиения: удалить полные дубли → отложить строки без цели → групповое разбиение 60/20/20 по паре (подкаст, эпизод). Разбиение целыми подкастами не выбрано: подкастов всего 48, а задача — прогноз для новых эпизодов уже известных подкастов, а не для новых подкастов. Временного столбца в данных нет, поэтому хронологическое разбиение невозможно; объекты считаются независимыми с точностью до группы.

8.3. Разбиение на train / valid / test

1. Удаляются полные дубли (остаётся строка с минимальным `row_id`).
2. Отделяются строки без целевой переменной — на них нельзя ни обучать, ни оценивать (в эксплуатации это как раз «новые» эпизоды, для которых нужен прогноз).
3. Разбиение 60/20/20 выполняется **по группам** — паре (подкаст, эпизод) — через SHA-256 от ключа и seed. Все строки одной пары попадают в одну часть; результат не зависит от порядка партиций и воспроизводится при повторном запуске.

```
pod_dedup = pod_raw.groupBy(*value_cols).agg(F.min("row_id").alias("row_id"))
pod_unlabelled = pod_dedup.filter(F.col("label").isNull())
pod_labelled = pod_dedup.filter(F.col("label").isNotNull())

group_hash = F.sha2(F.concat_ws(":", "podcast", "episode", F.lit(str(SEED))), 256)
bucket = F.pmod(F.conv(F.substring(group_hash, 1, 8), 16, 10).cast("long"), F.lit(100))
pod_split = (pod_labelled
    .withColumn("bucket", bucket)
    .withColumn("split", F.when(F.col("bucket") < 60, "train")
                         .when(F.col("bucket") < 80, "valid")
                         .otherwise("test"))
```

```
    .drop("bucket")
    .cache())

print("После удаления дублей:", pod_dedup.count(), "| без цели:", pod_unlabelled.count(),
      "| с целью:", pod_split.count())
(pod_split.groupBy("split")
    .agg(F.count("*").alias("rows"),
         F.countDistinct("podcast", "episode").alias("pairs"),
         F.countDistinct("podcast").alias("podcasts"),
         F.round(F.avg("label"), 2).alias("mean_label"),
         F.round(F.stddev("label"), 2).alias("std_label"),
         F.round(F.avg(F.col("length").isNull().cast("double")), 3).alias("length_null_share"))
    .orderBy(F.expr("CASE split WHEN 'train' THEN 0 WHEN 'valid' THEN 1 ELSE 2 END"))
    .show())

crossing = (pod_split.groupBy("podcast", "episode").agg(F.countDistinct("split").alias("parts"))
            .filter("parts > 1").count())
print("Пар, попавших в несколько частей:", crossing)
assert crossing == 0

pod_train = pod_split.filter("split = 'train'").drop("split").cache()
pod_valid = pod_split.filter("split = 'valid'").drop("split").cache()
pod_test = pod_split.filter("split = 'test'").drop("split").cache()
print("train / valid / test:", pod_train.count(), pod_valid.count(), pod_test.count())
```

```
После удаления дублей: 50000 | без цели: 5132 | с целью: 44868
+-----+-----+-----+--------+----------+---------+-----------------+
|split| rows|pairs|podcasts|mean_label|std_label|length_null_share|
+-----+-----+-----+--------+----------+---------+-----------------+
|train|26797| 2872|      48|     45.63|    27.09|            0.102|
|valid| 9016|  972|      48|     45.54|    27.22|            0.097|
| test| 9055|  955|      48|     45.27|    27.22|            0.097|
+-----+-----+-----+--------+----------+---------+-----------------+

Пар, попавших в несколько частей: 0
train / valid / test: 26797 9016 9055
```

После удаления 2 500 дублей осталось 50 000 строк; 5 132 из них без цели (резерв для «боевого» прогноза), 44 868 используются в работе: train 26 797 строк / 2 872 пары, valid 9 016 / 972, test 9 055 / 955. Ни одна пара не попала в две части (проверка `assert`), все 48 подкастов присутствуют в каждой части, а среднее (≈45,5 мин), разброс цели (≈27 мин) и доля пропусков длительности (≈10 %) в частях практически совпадают — разбиение сбалансировано.

8.4. Распределения предикторов и целевой переменной

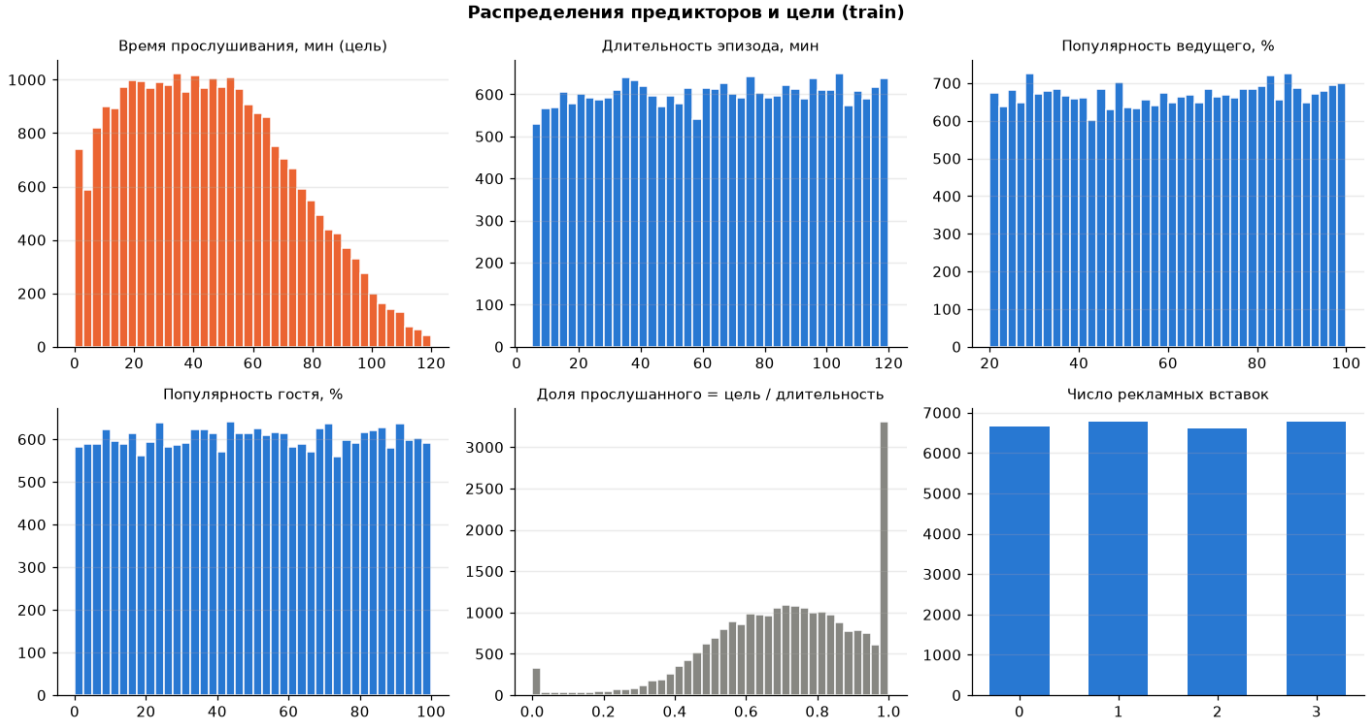
Разведочный анализ выполняется **только на train**, чтобы наблюдения valid/test не влияли на выбор признаков. Агрегаты считаются в Spark, в pandas переносятся только числовые столбцы train (≈28 тыс. строк × 6 столбцов) для гистограмм.

```
eda = (pod_train
       .select("label", "length", "host_pop", "guest_pop", "ads",
               (F.col("label") / F.col("length")).alias("listen_share"))
       .toPandas())
print(eda.describe().T.round(2).to_string())

fig, axes = plt.subplots(2, 3, figsize=(13, 7))
spec = [("label", "Время прослушивания, мин (цель)", C2), ("length", "Длительность эпизода, мин", C1),
        ("host_pop", "Популярность ведущего, %", C1), ("guest_pop", "Популярность гостя, %", C1),
        ("listen_share", "Доля прослушанного = цель / длительность", GRAY)]
for ax, (col, title, color) in zip(axes.flat, spec):
    ax.hist(eda[col].dropna(), bins=40, color=color, edgecolor="white")
    ax.set_title(title, fontsize=10, fontweight="normal"); ax.grid(axis="x", visible=False)
ads = eda["ads"].value_counts().sort_index()
```

```
axes.flat[5].bar(ads.index.astype(str), ads.values, color=C1, width=0.6)
axes.flat[5].set_title("Число рекламных вставок", fontsize=10, fontweight="normal")
axes.flat[5].grid(axis="x", visible=False)
fig.suptitle("Распределения предикторов и цели (train)", fontweight="bold")
plt.tight_layout(); plt.savefig(OUT / "self_distributions.png", dpi=160); plt.show()
```

	count	mean	std	min	25%	50%	75%	max
label	26797.0	45.63	27.09	0.00	23.40	43.69	64.96	119.97
length	24063.0	63.07	33.08	5.00	34.56	63.33	91.79	120.00
host_pop	26797.0	60.24	23.21	20.00	39.93	60.39	80.50	99.99
guest_pop	24120.0	50.15	28.79	0.01	25.25	50.12	75.11	100.00
ads	26797.0	1.50	1.12	0.00	1.00	1.00	3.00	3.00
listen_share	24063.0	0.71	0.21	0.00	0.58	0.73	0.88	1.00




```
print("Корреляция Пирсона с целью (train, попарно по непустым):")
print(eda.corr()["label"].drop("label").round(3).to_string())

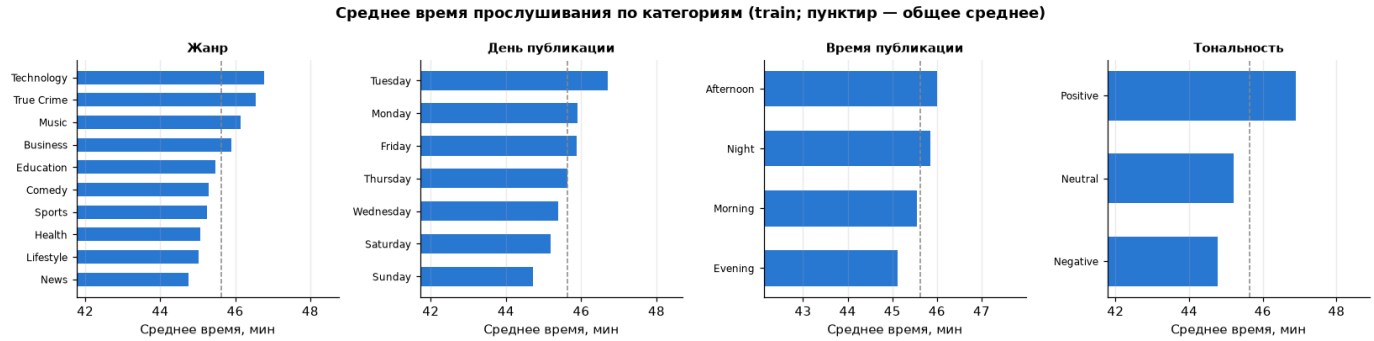
fig, axes = plt.subplots(1, 4, figsize=(15, 3.8))
for ax, col, title in zip(axes, ["genre", "day", "time_of_day", "sentiment"],
                          ["Жанр", "День публикации", "Время публикации", "Тональность"]):
    g = (pod_train.groupBy(col).agg(F.avg("label").alias("mean"), F.count("*").alias("n"))
         .orderBy("mean").toPandas())
    ax.barh(g[col], g["mean"], color=C1, height=0.6)
    ax.axvline(eda["label"].mean(), color=GRAY, linestyle="--", linewidth=1)
    ax.set_title(title, fontsize=10); ax.grid(axis="y", visible=False)
    ax.set_xlabel("Среднее время, мин")
    ax.set_xlim(g["mean"].min() - 3, g["mean"].max() + 2)
    ax.tick_params(axis="y", labelleft=True, labelsize=8)
fig.suptitle("Среднее время прослушивания по категориям (train; пунктир — общее среднее)",
fontweight="bold")
plt.tight_layout(); plt.savefig(OUT / "self_categories.png", dpi=160); plt.show()

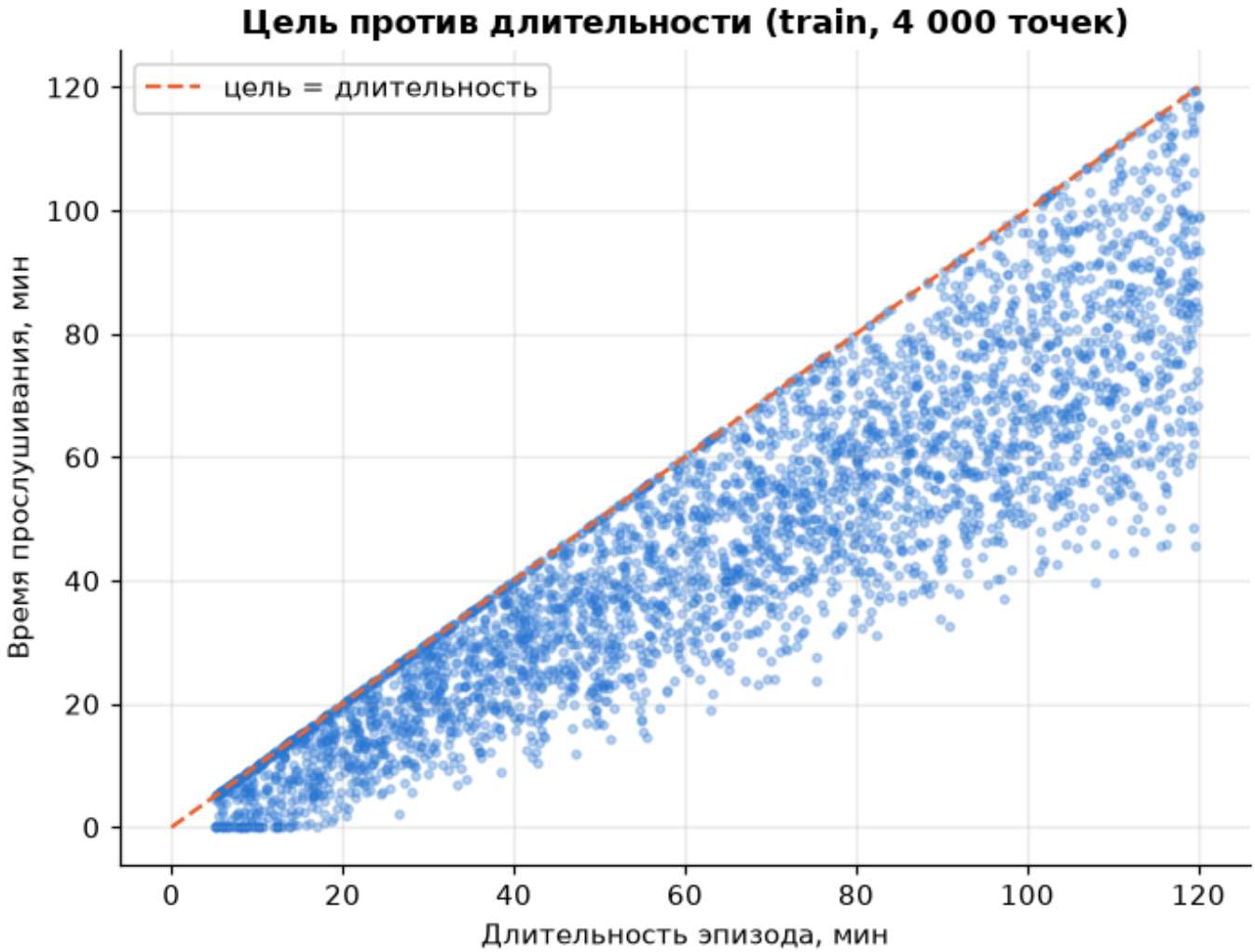
sample = eda.sample(4000, random_state=SEED)
fig, ax = plt.subplots(figsize=(6.5, 5))
ax.scatter(sample["length"], sample["label"], s=8, alpha=0.3, color=C1)
ax.plot([0, 120], [0, 120], "--", color=C2, label="цель = длительность")
ax.set(xlabel="Длительность эпизода, мин", ylabel="Время прослушивания, мин",
       title="Цель против длительности (train, 4 000 точек)")
ax.legend()
plt.tight_layout(); plt.savefig(OUT / "self_length_scatter.png", dpi=160); plt.show()

miss = (pod_train.agg(*[F.avg(F.col(c).isNull().cast("double")).alias(c) for c in ["length", "guest_pop"]])
        .first().asDict())
```

```
print("Доля пропусков в train:", {k: round(v, 3) for k, v in miss.items()})
print("Среднее цели при пропуске / наличии length:",
      [round(r["m"], 2) for r in pod_train.groupBy(F.col("length").isNull().alias("na"))
       .agg(F.avg("label").alias("m")).orderBy("na").collect()])
print("Среднее цели при пропуске / наличии guest_pop:",
      [round(r["m"], 2) for r in pod_train.groupBy(F.col("guest_pop").isNull().alias("na"))
       .agg(F.avg("label").alias("m")).orderBy("na").collect()])
```

```
Корреляция Пирсона с целью (train, попарно по непустым):
length          0.901
host_pop        0.044
guest_pop      -0.009
ads            -0.081
listen_share    0.423
```






```
Доля пропусков в train: {'length': 0.102, 'guest_pop': 0.1}
Среднее цели при пропуске / наличии length: [45.69, 45.12]
Среднее цели при пропуске / наличии guest_pop: [45.62, 45.71]
```

Что показали распределения (train).

- Цель распределена от 0 до 120 мин, среднее 45,6, медиана 43,7, без тяжёлых хвостов — логарифмирование не нужно; RMSE и MAE интерпретируются в минутах.
- Длительность эпизода почти равномерна в диапазоне 5–120 мин; популярность ведущего (20–100) и гостя (0–100) тоже близки к равномерным; 0–3 рекламные вставки встречаются одинаково часто.
- Главная связь — с длительностью: корреляция 0,90; на диаграмме рассеяния все точки лежат ниже линии «цель = длительность» (дослушать дольше эпизода нельзя), доля прослушанного в среднем 0,71; около 11,5 % эпизодов дослушаны целиком (пик доли в точке 1,0 — цель усечена длительностью), ≈1 % — с нулевым временем. Остальные числовые признаки почти не коррелируют с целью: реклама −0,08, ведущий +0,04, гость ≈ 0.
- Категории дают различия лишь в 1–2 минуты: Technology и True Crime дольше, News и Lifestyle короче; позитивная тональность ≈ +2 мин к негативной; вторник — самый «долгий» день.
- Пропуск длительности почти не меняет среднюю цель (45,1 против 45,7 мин), но для таких строк исчезает главный предиктор — на них ожидается максимальная ошибка. Пропуск популярности гостя на цель не влияет.

Вывод: качество будет определяться в основном длительностью; простое правило «средняя доля × длительность» должно быть сильным базовым соперником.

8.5. Выбор предикторов, целевой переменной и точки возможной утечки

Целевая переменная: `label` = `Listening_Time_minutes` — непрерывная величина в минутах, поэтому задача — регрессия, метрики — RMSE (основная), MAE, R^2.

Поле	Решение	Обоснование
`length` (длительность)	признак + медианная импутация + флаг пропуска	известна в момент публикации; главный ожидаемый предиктор
`host_pop`, `ads`, `day`, `time_of_day`, `sentiment`, `genre`	признаки	известны до публикации
`guest_pop`	признак + медианная импутация + флаг пропуска	пропуск может означать «гостя нет» — флаг сохраняет эту информацию
`podcast`	категориальный признак (one-hot)	все 48 подкастов есть во всех частях, словарь обучается на train
`episode` («Episode N»)	**не используется**	технический номер, не несёт смысла; используется только как ключ группового разбиения
`row_id`	**не используется**	технический идентификатор

Точки возможной утечки и меры против неё:

1. **Дубли и повторяющиеся ключи.** 2 500 полных копий строк и ~11 строк на пару (подкаст, эпизод): при построчном случайном разбиении копии оказались бы и в train, и в test, и модель «узнавала» бы ответ. Мера — удаление дублей и групповое разбиение по ключу (п. 8.3).
2. **Статистики предобработки.** Медианы для импутации, словари категорий и стандартные отклонения считаются внутри `Pipeline` только на train (`Imputer`, `StringIndexer`, `StandardScaler`), а не по всему файлу.

3. **Признаки, производные от цели.** Доля прослушивания `label / length` и целевое кодирование подкаста средним временем по всему набору напрямую содержат ответ — в признаки они не входят. Доля прослушивания используется только как базовое правило, и её среднее берётся только по train.
4. **Момент наблюдения.** `host_pop` и `guest_pop` могли измеряться уже после выхода эпизода (популярность растёт вместе с прослушиваниями). В наборе нет времени их фиксации — это ограничение данных; в эксплуатации нужно брать значения на момент публикации.
5. **Выбор модели по test.** Гиперпараметры и модель выбираются только по valid; test используется один раз.

```
POD_CAT = ["podcast", "genre", "day", "time_of_day", "sentiment"]
POD_NUM = ["length_imp", "host_pop", "guest_imp", "ads", "length_missing", "guest_missing"]

def pod_pipeline(kind, **params):
    flags = SQLTransformer(statement=(
        "SELECT *, CAST(length IS NULL AS DOUBLE) AS length_missing, "
        "CAST(guest_pop IS NULL AS DOUBLE) AS guest_missing FROM __THIS__"))
    imputer = Imputer(inputCols=["length", "guest_pop"], outputCols=["length_imp", "guest_imp"],
                      strategy="median")
    index_cols = [c + "_idx" for c in POD_CAT]
    encoded_cols = [c + "_ohe" for c in POD_CAT]
    indexer = StringIndexer(inputCols=POD_CAT, outputCols=index_cols,
                            handleInvalid="keep", stringOrderType="alphabetAsc")
    encoder = OneHotEncoder(inputCols=index_cols, outputCols=encoded_cols,
                            handleInvalid="keep", dropLast=False)
    stages = [flags, imputer, indexer, encoder]
    if kind == "LINEAR":
        stages += [
            VectorAssembler(inputCols=POD_NUM + encoded_cols, outputCol="features_raw"),
            StandardScaler(inputCol="features_raw", outputCol="features", withMean=False, withStd=True),
            LinearRegression(maxIter=200, standardization=False,
                             regParam=params.get("regParam", 0.0),
                             elasticNetParam=params.get("elasticNetParam", 0.0)),
        ]
    elif kind == "RF":
        stages += [
            VectorAssembler(inputCols=POD_NUM + encoded_cols, outputCol="features"),
            RandomForestRegressor(seed=SEED, maxBins=64, featureSubsetStrategy="onethird",
                                  numTrees=params.get("numTrees", 60), maxDepth=params.get("maxDepth", 8),
                                  minInstancesPerNode=params.get("minInstancesPerNode", 5)),
        ]
    elif kind == "GBT":
        stages += [
            VectorAssembler(inputCols=POD_NUM + encoded_cols, outputCol="features"),
            GBTRegressor(seed=SEED, maxBins=64, stepSize=params.get("stepSize", 0.1),
                         maxIter=params.get("maxIter", 50), maxDepth=params.get("maxDepth", 5),
                         subsamplingRate=0.8),
        ]
    else:
        raise ValueError(kind)
    return Pipeline(stages=stages)

# Базовые правила (статистики только по train)
pod_train_mean = pod_train.agg(F.avg("label")).first()[0]
share_stats = pod_train.filter(F.col("length").isNotNull()).agg(
    F.avg(F.col("label") / F.col("length")).alias("share")).first()
POD_SHARE = share_stats["share"]
pod_mean_no_length = pod_train.filter(F.col("length").isNull()).agg(F.avg("label")).first()[0]

def pod_baseline(df, rule):
    if rule == "mean":
        return df.withColumn("prediction", F.lit(pod_train_mean))
```

```
    if rule == "share":   # средняя доля прослушивания × длительность эпизода
        return df.withColumn("prediction", F.when(F.col("length").isNotNull(),
                                F.col("length") * F.lit(POD_SHARE)).otherwise(F.lit(pod_mean_no_length)))
    raise ValueError(rule)

print(f"Среднее train = {pod_train_mean:.2f} мин; средняя доля прослушивания = {POD_SHARE:.3f}; "
      f"среднее при пропущенной длительности = {pod_mean_no_length:.2f} мин")
```

```
Среднее train = 45.63 мин; средняя доля прослушивания = 0.714; среднее при пропущенной длительности = 45.12
мин
```

8.6. Обучение моделей и подбор гиперпараметров

Подбор выполняется **минимизацией RMSE на valid** по небольшим сеткам для трёх семейств: линейная регрессия (сила и тип регуляризации), случайный лес (число и глубина деревьев), градиентный бустинг GBT (глубина и число итераций). Для линейной модели дополнительно выполнена 3-кратная кросс-валидация на train (`CrossValidator`) — чтобы сравнить два способа подбора.

```
grids = {
    "LINEAR": [dict(regParam=r, elasticNetParam=e) for r in [0.0, 0.01, 0.1, 1.0] for e in [0.0, 1.0]
               if not (r == 0.0 and e == 1.0)],
    "RF": [dict(numTrees=t, maxDepth=d) for t in [60, 120] for d in [6, 10, 14]],
    "GBT": [dict(maxDepth=d, maxIter=i) for d in [3, 5, 7] for i in [40, 100]],
}
FAMILY = {"LINEAR": "Линейная регрессия", "RF": "Случайный лес", "GBT": "Градиентный бустинг (GBT)"}

tuning_rows = []
for kind, grid in grids.items():
    for params in grid:
        sc.setJobGroup("P5_SELF_" + kind, f"{FAMILY[kind]} {params}")
        t0 = perf_counter()
        model = pod_pipeline(kind, **params).fit(pod_train)
        fit_s = perf_counter() - t0
        m = regression_metrics(model.transform(pod_valid))
        tuning_rows.append({"kind": kind, "params": params, "fit_s": fit_s, **m, "_model": model})
        print(f"{FAMILY[kind]:26s} {str(params):40s} RMSE={m['rmse']:.3f} MAE={m['mae']:.3f} R2=
{m['r2']:.4f} ({fit_s:.1f} c)")

tuning = pd.DataFrame(tuning_rows)
```

```
Линейная регрессия         {'regParam': 0.0, 'elasticNetParam': 0.0} RMSE=13.543 MAE=10.126 R2=0.7524 (4.9
c)
Линейная регрессия         {'regParam': 0.01, 'elasticNetParam': 0.0} RMSE=13.543 MAE=10.126 R2=0.7524 (3.5
c)
Линейная регрессия         {'regParam': 0.01, 'elasticNetParam': 1.0} RMSE=13.541 MAE=10.123 R2=0.7525 (4.3
c)
Линейная регрессия         {'regParam': 0.1, 'elasticNetParam': 0.0} RMSE=13.544 MAE=10.126 R2=0.7523 (3.5
c)
Линейная регрессия         {'regParam': 0.1, 'elasticNetParam': 1.0} RMSE=13.535 MAE=10.111 R2=0.7527 (2.8
c)
Линейная регрессия         {'regParam': 1.0, 'elasticNetParam': 0.0} RMSE=13.574 MAE=10.149 R2=0.7512 (3.3
c)
Линейная регрессия         {'regParam': 1.0, 'elasticNetParam': 1.0} RMSE=13.714 MAE=10.215 R2=0.7461 (2.7
c)
Случайный лес              {'numTrees': 60, 'maxDepth': 6}          RMSE=14.554 MAE=11.229 R2=0.7140 (8.8 c)
Случайный лес              {'numTrees': 60, 'maxDepth': 10}         RMSE=13.602 MAE=10.241 R2=0.7502 (23.9
c)
Случайный лес              {'numTrees': 60, 'maxDepth': 14}         RMSE=13.521 MAE=10.085 R2=0.7532 (76.5
c)
Случайный лес              {'numTrees': 120, 'maxDepth': 6}         RMSE=14.620 MAE=11.321 R2=0.7114 (14.5
c)
Случайный лес              {'numTrees': 120, 'maxDepth': 10}        RMSE=13.573 MAE=10.202 R2=0.7513 (49.5
c)
```

```
Случайный лес              {'numTrees': 120, 'maxDepth': 14}        RMSE=13.481 MAE=10.050 R2=0.7547 (142.8
с)
Градиентный бустинг (GBT)  {'maxDepth': 3, 'maxIter': 40}           RMSE=13.479 MAE=10.040 R2=0.7547 (33.2
с)
Градиентный бустинг (GBT)  {'maxDepth': 3, 'maxIter': 100}          RMSE=13.431 MAE=9.990 R2=0.7565 (39.3 с)
Градиентный бустинг (GBT)  {'maxDepth': 5, 'maxIter': 40}           RMSE=13.557 MAE=10.062 R2=0.7518 (25.5
с)
Градиентный бустинг (GBT)  {'maxDepth': 5, 'maxIter': 100}          RMSE=13.683 MAE=10.128 R2=0.7472 (140.8
с)
Градиентный бустинг (GBT)  {'maxDepth': 7, 'maxIter': 40}           RMSE=13.926 MAE=10.303 R2=0.7382 (85.8
с)
Градиентный бустинг (GBT)  {'maxDepth': 7, 'maxIter': 100}          RMSE=14.230 MAE=10.490 R2=0.7266 (253.0
с)
```

```python
# Лучшая конфигурация каждого семейства по RMSE valid + базовые правила
best_per_family = tuning.loc[tuning.groupby("kind")["rmse"].idxmin()].set_index("kind")
pod_models = {FAMILY[k]: best_per_family.loc[k, "_model"] for k in best_per_family.index}

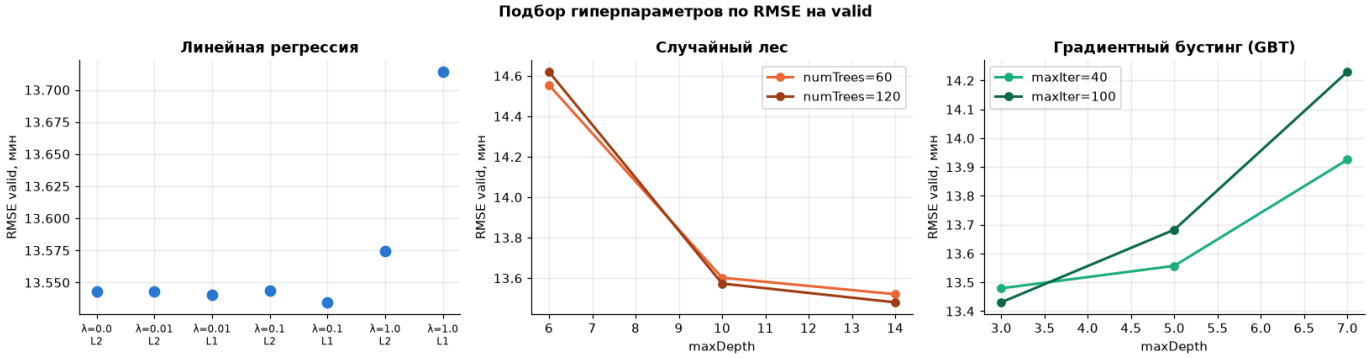
summary_rows = []
for k in ["LINEAR", "RF", "GBT"]:
    row = best_per_family.loc[k]
    train_m = regression_metrics(row["_model"].transform(pod_train))
    summary_rows.append({"Решение": FAMILY[k], "Гиперпараметры": str(row["params"]),
                         "RMSE train": train_m["rmse"], "RMSE valid": row["rmse"],
                         "MAE valid": row["mae"], "R² valid": row["r2"], "fit, c": row["fit_s"]})
for rule, title in [("mean", "Базовое: среднее train"), ("share", "Базовое: доля × длительность")]:
    m = regression_metrics(pod_baseline(pod_valid, rule))
    summary_rows.append({"Решение": title, "Гиперпараметры": "—", "RMSE train":
                         regression_metrics(pod_baseline(pod_train, rule))["rmse"],
                         "RMSE valid": m["rmse"], "MAE valid": m["mae"], "R² valid": m["r2"], "fit, c":
0.0})
pod_validation = pd.DataFrame(summary_rows).sort_values("RMSE valid")
print(pod_validation.round(3).to_string(index=False))

learned = pod_validation[pod_validation["Решение"].isin(pod_models)]
BEST_POD_NAME = str(learned.iloc[0]["Решение"])
BEST_POD = pod_models[BEST_POD_NAME]
print("\nВыбрана модель:", BEST_POD_NAME, "|", learned.iloc[0]["Гиперпараметры"])

# Кривые подбора
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
lin = tuning[tuning["kind"] == "LINEAR"].copy()
lin["label"] = lin["params"].apply(lambda p: f"λ={p['regParam']}\n{'L1' if p['elasticNetParam'] else
'L2'}")
# точечная диаграмма: различия малы, а столбцы с ненулевой базой исказили бы пропорции
axes[0].plot(lin["label"], lin["rmse"], "o", color=C1, markersize=8)
axes[0].set(title="Линейная регрессия", ylabel="RMSE valid, мин"); axes[0].tick_params(axis="x",
labelsize=8)
for ax, kind, series_key, x_key, colors in [(axes[1], "RF", "numTrees", "maxDepth", [C2, "#a23d14"]),
                                            (axes[2], "GBT", "maxIter", "maxDepth", [C3, "#0d6b4a"])]:
    part = tuning[tuning["kind"] == kind]
    for (val, grp), color in zip(part.groupby(part["params"].apply(lambda p: p[series_key])), colors):
        xs = grp["params"].apply(lambda p: p[x_key])
        ax.plot(xs, grp["rmse"], marker="o", color=color, linewidth=2, label=f"{series_key}={val}")
    ax.set(title=FAMILY[kind], xlabel=x_key, ylabel="RMSE valid, мин"); ax.legend()
fig.suptitle("Подбор гиперпараметров по RMSE на valid", fontweight="bold")
plt.tight_layout(); plt.savefig(OUT / "self_tuning.png", dpi=160); plt.show()
```

```
                    Решение                                Гиперпараметры  RMSE train  RMSE valid  MAE valid
R² valid  fit, c
   Градиентный бустинг (GBT)          {'maxDepth': 3, 'maxIter': 100}      13.402      13.431      9.990
0.756  39.254
              Случайный лес         {'numTrees': 120, 'maxDepth': 14}      11.843      13.481     10.050
0.755 142.812
         Линейная регрессия {'regParam': 0.1, 'elasticNetParam': 1.0}      13.800      13.535     10.111
0.753   2.804
Базовое: доля × длительность                                       –      14.174      13.942     10.390
0.738   0.000
      Базовое: среднее train                                       –      27.090      27.216     22.671
-0.000   0.000

Выбрана модель: Градиентный бустинг (GBT) | {'maxDepth': 3, 'maxIter': 100}
```




```
# Кросс-валидация для линейной модели (3 фолда на train): все преобразования переобучаются внутри фолдов
lin_pipe = pod_pipeline("LINEAR")
lin_est = lin_pipe.getStages()[-1]
cv_grid = (ParamGridBuilder()
           .addGrid(lin_est.regParam, [0.0, 0.01, 0.1, 1.0])
           .addGrid(lin_est.elasticNetParam, [0.0, 1.0])
           .build())
cv = CrossValidator(estimator=lin_pipe, estimatorParamMaps=cv_grid,
                    evaluator=RegressionEvaluator(labelCol="label", metricName="rmse"),
                    numFolds=3, seed=SEED, parallelism=2)
sc.setJobGroup("P5_SELF_CV", "CrossValidator для линейной регрессии")
t0 = perf_counter()
cv_model = cv.fit(pod_train)
print(f"CrossValidator: {len(cv_grid)} конфигураций × 3 фолда за {perf_counter() - t0:.1f} с")
cv_table = pd.DataFrame([{"regParam": pm[lin_est.regParam], "elasticNet": pm[lin_est.elasticNetParam],
"RMSE CV": r}
                         for pm, r in zip(cv_grid, cv_model.avgMetrics)]).sort_values("RMSE CV")
print(cv_table.round(4).to_string(index=False))
best_cv = cv_model.bestModel.stages[-1]
print("Лучшее по CV: regParam =", best_cv.getRegParam(), ", elasticNet =", best_cv.getElasticNetParam(),
      "| RMSE valid =", round(regression_metrics(cv_model.bestModel.transform(pod_valid))["rmse"], 3))
```

```
CrossValidator: 8 конфигураций × 3 фолда за 58.6 с
 regParam  elasticNet  RMSE CV
     0.10         1.0  13.8202
     0.01         1.0  13.8319
     0.01         0.0  13.8347
     0.00         1.0  13.8347
     0.00         0.0  13.8347
     0.10         0.0  13.8349
     1.00         0.0  13.8585
     1.00         1.0  13.9670
Лучшее по CV: regParam = 0.1 , elasticNet = 1.0 | RMSE valid = 13.535
```

Результат подбора (valid).

- **Линейная регрессия** почти нечувствительна к регуляризации: RMSE 13,54 при λ ≤ 0,1, лучшая — λ = 0,1 с L1 (13,535); сильная регуляризация λ = 1 ухудшает (до 13,71). Кросс-валидация на train выбрала ту же конфигурацию (λ = 0,1, L1; RMSE CV 13,82) — оба способа подбора согласуются.

- **Случайный лес** улучшается с глубиной: 14,55–14,62 при depth 6 → 13,48 при depth 14 и 120 деревьях; число деревьев влияет мало. Разрыв RMSE train 11,84 / valid 13,48 показывает начало переобучения, дальнейший рост глубины смысла не имеет.
- **GBT** — обратная картина: лучше всего неглубокие деревья (depth 3, 100 итераций: 13,43), а при depth 7 ошибка растёт до 14,23 — бустинг глубокими деревьями переобучается. Train/valid у лучшего GBT 13,40 / 13,43 — переобучения нет.

Все три модели лучше базовых правил: «среднее train» — 27,22 (R^2 = 0), «доля × длительность» — 13,94 (R^2 0,738). Однако выигрыш над сильным правилом невелик: лучший GBT снижает RMSE на 0,51 мин (3,7 %) и MAE на 0,40 мин. Выбран **GBT (maxDepth = 3, maxIter = 100)** с минимальной RMSE на valid.

8.7. Однократная оценка на test и визуализация

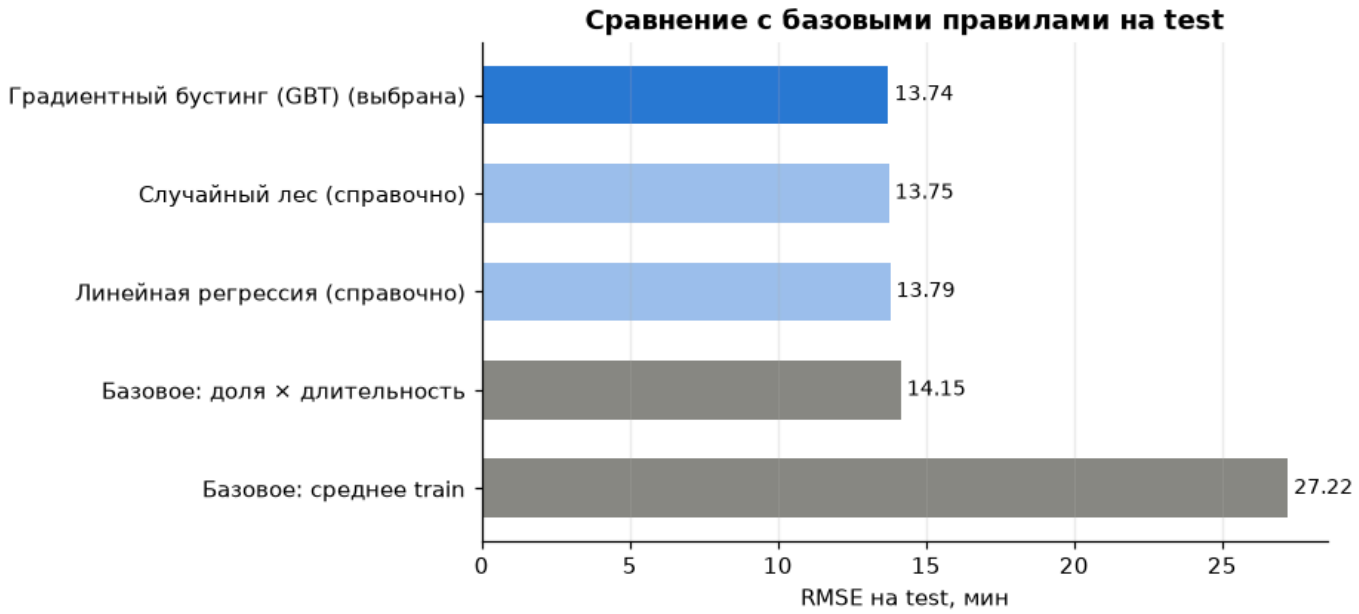
Выбранная по valid модель применяется к test один раз; базовые правила — фиксированные формулы со статистиками train. Остальные модели пересчитаны на test только справочно, после фиксации выбора.

```
sc.setJobGroup("P5_SELF_TEST", "Самостоятельная: test")
pod_test_pred = (BEST_POD.transform(pod_test)
    .select("row_id", "podcast", "genre", "length", "label", "prediction")
    .withColumn("residual", F.col("label") - F.col("prediction"))
    .cache())
pod_test_metrics = regression_metrics(pod_test_pred)
print("TEST", BEST_POD_NAME, {k: round(v, 4) for k, v in pod_test_metrics.items()})

final_rows = []
for name, model in pod_models.items():
    m = pod_test_metrics if name == BEST_POD_NAME else regression_metrics(model.transform(pod_test))
    final_rows.append({"Решение": name + (" (выбрана)" if name == BEST_POD_NAME else " (справочно)"), **m})
for rule, title in [("mean", "Базовое: среднее train"), ("share", "Базовое: доля × длительность")]:
    final_rows.append({"Решение": title, **regression_metrics(pod_baseline(pod_test, rule))})
pod_final = pd.DataFrame(final_rows).sort_values("rmse")
print("\nМетрики на test:")
print(pod_final.round(3).to_string(index=False))

fig, ax = plt.subplots(figsize=(8.5, 4))
colors = [C1 if "выбрана" in n else (GRAY if "Базовое" in n else "#9cc0ec") for n in pod_final["Решение"]]
bars = ax.barh(pod_final["Решение"][::-1], pod_final["rmse"][::-1], color=colors[::-1], height=0.6)
for b, val in zip(bars, pod_final["rmse"][::-1]):
    ax.text(val + 0.2, b.get_y() + b.get_height() / 2, f"{val:.2f}", va="center", fontsize=9)
ax.set(xlabel="RMSE на test, мин", title="Сравнение с базовыми правилами на test")
ax.grid(axis="y", visible=False)
plt.tight_layout(); plt.savefig(OUT / "self_test_rmse.png", dpi=160); plt.show()
```

```
TEST Градиентный бустинг (GBT) {'rmse': 13.7356, 'mae': 10.19, 'r2': 0.7453, 'negative_share': 0.0}
Метрики на test:
                          Решение   rmse    mae     r2  negative_share
Градиентный бустинг (GBT) (выбрана) 13.736 10.190  0.745           0.000
          Случайный лес (справочно) 13.748 10.209  0.745           0.000
     Линейная регрессия (справочно) 13.790 10.269  0.743           0.001
       Базовое: доля × длительность 14.154 10.518  0.730           0.000
             Базовое: среднее train 27.217 22.678 -0.000           0.000
```



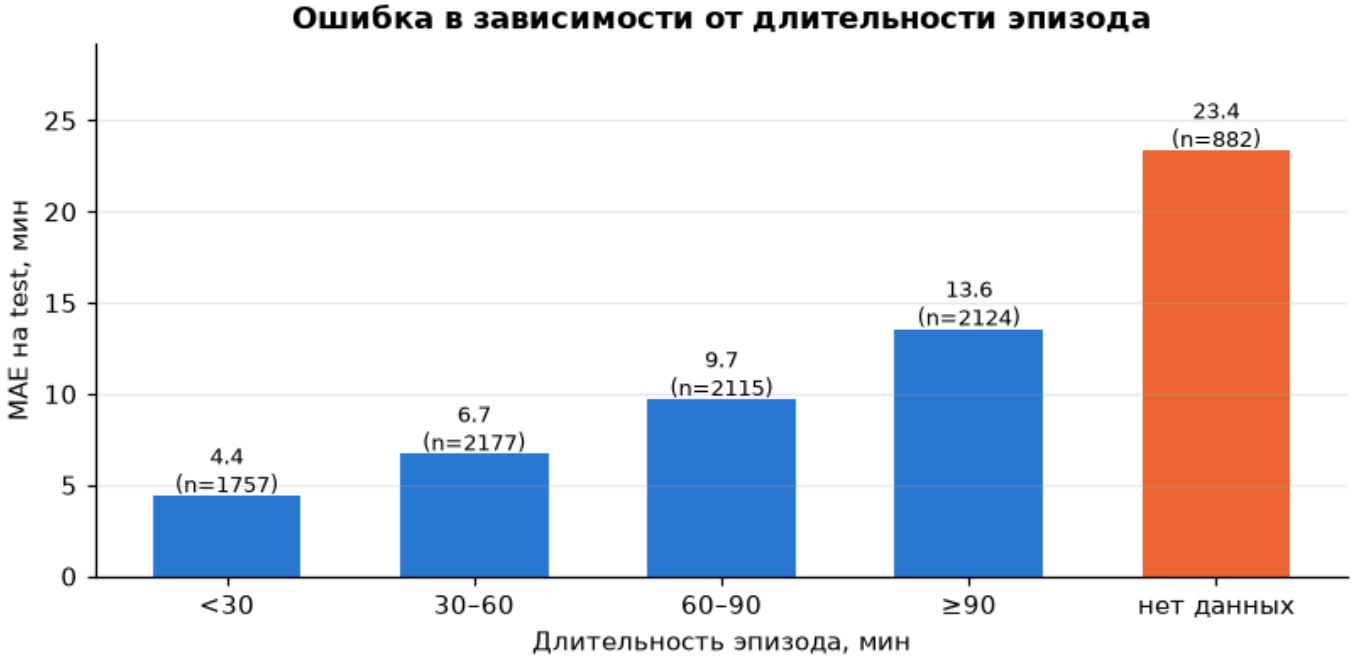

```
loc = pod_test_pred.toPandas()     # test ≈ 9 тыс. строк × 6 столбцов
s = loc.sample(4000, random_state=SEED)
fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
axes[0].scatter(s["label"], s["prediction"], s=8, alpha=0.3, color=C1)
axes[0].plot([0, 120], [0, 120], "--", color=C2, label="y = ŷ")
axes[0].set(xlabel="Фактическое время, мин", ylabel="Прогноз, мин", title=f"Факт и прогноз на test
({BEST_POD_NAME})")
axes[0].legend()
axes[1].hist(loc["residual"], bins=60, color=C1, edgecolor="white")
axes[1].axvline(0, color=C2, linestyle="--")
axes[1].set(xlabel="Остаток = факт − прогноз, мин", ylabel="Число эпизодов", title="Распределение остатков
на test")
axes[1].grid(axis="x", visible=False)
plt.tight_layout(); plt.savefig(OUT / "self_test_scatter.png", dpi=160); plt.show()

# Ошибка по группам длительности (включая пропуск длительности)
by_len = (pod_test_pred
    .withColumn("length_group", F.when(F.col("length").isNull(), "нет данных")
        .when(F.col("length") < 30, "<30").when(F.col("length") < 60, "30–60")
        .when(F.col("length") < 90, "60–90").otherwise("≥90"))
    .groupBy("length_group").agg(F.count("*").alias("n"), F.avg(F.abs("residual")).alias("mae"),
                                 F.avg("residual").alias("bias"))
    .toPandas().set_index("length_group").loc[["<30", "30–60", "60–90", "≥90", "нет данных"]])
print(by_len.round(2).to_string())
fig, ax = plt.subplots(figsize=(7.5, 3.8))
bars = ax.bar(by_len.index, by_len["mae"], color=[C1] * 4 + [C2], width=0.6)
for b, (val, n) in zip(bars, zip(by_len["mae"], by_len["n"])):
    ax.text(b.get_x() + b.get_width() / 2, val + 0.2, f"{val:.1f}\n(n={n})", ha="center", fontsize=8.5)
ax.set(xlabel="Длительность эпизода, мин", ylabel="MAE на test, мин",
       title="Ошибка в зависимости от длительности эпизода", ylim=(0, by_len["mae"].max() * 1.25))
ax.grid(axis="x", visible=False)
plt.tight_layout(); plt.savefig(OUT / "self_mae_by_length.png", dpi=160); plt.show()
```





length_group	n	mae	bias
<30	1757	4.40	-0.14
30–60	2177	6.70	-0.22
60–90	2115	9.70	0.23
≥90	2124	13.57	0.05
нет данных	882	23.36	0.69




```
# Важность признаков выбранной модели (для деревьев) или коэффициенты (для линейной)
last = BEST_POD.stages[-1]
if hasattr(last, "featureImportances"):
    names = vector_names(BEST_POD, pod_valid, "features")
    imp = pd.Series(last.featureImportances.toArray(), index=names).sort_values(ascending=False)
    xlabel = "Impurity-based importance"
else:
    names = vector_names(BEST_POD, pod_valid, "features_raw")
    imp = pd.Series(np.abs(last.coefficients.toArray()), index=names).sort_values(ascending=False)
    xlabel = "|коэффициент| (масштабированные признаки)"
top = imp.head(12)[::-1]
fig, ax = plt.subplots(figsize=(7.5, 4.6))
ax.barh(top.index, top.values, color=C1, height=0.6)
ax.set(xlabel=xlabel, title=f"Наиболее важные признаки: {BEST_POD_NAME}")
ax.grid(axis="y", visible=False)
plt.tight_layout(); plt.savefig(OUT / "self_importance.png", dpi=160); plt.show()
print(imp.head(8).round(4).to_string())

# Сохранение модели и паспорта применения
```

```
BEST_POD.write().overwrite().save((OUT / "podcast_regressor").as_posix())
pod_config = {
    "seed": SEED, "task": "регрессия Listening_Time_minutes",
    "model": BEST_POD_NAME, "hyperparameters": str(learned.iloc[0]["Гиперпараметры"]),
    "split": "группы (podcast, episode), SHA-256 + seed, 60/20/20, после удаления полных дублей",
    "baseline_share": POD_SHARE, "test_rmse": pod_test_metrics["rmse"], "test_mae":
pod_test_metrics["mae"],
    "spark_version": spark.version,
}
(OUT / "podcast_config.json").write_text(json.dumps(pod_config, ensure_ascii=False, indent=2),
encoding="utf-8")
print(json.dumps(pod_config, ensure_ascii=False, indent=2))
```




```
length_imp                0.7483
ads                       0.0895
host_pop                  0.0545
length_missing            0.0245
guest_imp                 0.0142
sentiment_ohe_Positive    0.0138
genre_ohe_Technology      0.0039
genre_ohe_True Crime      0.0028
{
  "seed": 42,
  "task": "регрессия Listening_Time_minutes",
  "model": "Градиентный бустинг (GBT)",
  "hyperparameters": "{'maxDepth': 3, 'maxIter': 100}",
  "split": "группы (podcast, episode), SHA-256 + seed, 60/20/20, после удаления полных дублей",
  "baseline_share": 0.7138908059435125,
  "test_rmse": 13.73562227131814,
  "test_mae": 10.189957807985087,
  "spark_version": "3.5.3"
}
```

Итог на test. Выбранный GBT: RMSE 13,74 мин, MAE 10,19 мин, R^2 0,745 — почти как на valid (13,43), т.е. оценка устойчива и переобучения на этапе подбора нет. Модель лучше обоих базовых правил: «среднее train» 27,22 (вдвое хуже) и «доля × длительность» 14,15 (на 0,42 мин, или 3 %, хуже). Лес и линейная модель на test практически равны выбранной (13,75 и 13,79) — разница между семействами меньше, чем разница с базовым правилом.

Анализ ошибок. Остатки симметричны вокруг нуля, смещения нет. Ошибка растёт с длительностью эпизода: MAE 4,4 мин для эпизодов короче 30 мин и 13,6 мин для ≥ 90 мин (у длинного эпизода абсолютный разброс времени прослушивания больше). Для эпизодов **без длительности** MAE 23,4 мин — почти как у постоянного прогноза: медианная импутация не восстанавливает главный признак. Важность признаков GBT: длительность 75 %, реклама 9 %, популярность ведущего 5 %, флаг пропуска длительности 2,5 %; подкаст, день и время публикации почти не используются.

Выводы по самостоятельной части.

1. Ключевое решение — схема разбиения: удалены 2 500 полных дублей и выполнено групповое разбиение по паре (подкаст, эпизод); без этого метрику завышали бы копии строк.
2. Время прослушивания определяется прежде всего длительностью эпизода (в среднем дослушивают ≈71 %); остальные признаки дают поправку в 1–2 минуты.
3. Модели Spark MLlib (GBT, RF, LR) улучшают сильное содержательное правило всего на ≈3 %, а постоянный прогноз — вдвое. Более сложная модель здесь оправдана слабо: линейная регрессия обучается за 2 с против 80 с у GBT при разнице RMSE на test 0,05 мин.
4. Для улучшения важнее данные, чем алгоритм: заполнить пропуски длительности из каталога эпизодов (сейчас на них ошибка 23 мин), добавить историю прослушиваний подкаста до публикации, зафиксировать момент измерения популярности ведущего и гостя.

7. Ответы на контрольные вопросы

1. Чем параметр модели отличается от гиперпараметра? Параметры подбираются алгоритмом по train при `fit`: коэффициенты и свободный член логистической/линейной регрессии; пороги разбиений и значения в листьях деревьев случайного леса; координаты центров KMeans. Гиперпараметры задаёт аналитик до обучения: `regParam`, `elasticNetParam`, `maxIter` для LR; `numTrees`, `maxDepth`, `maxBins`, `featureSubsetStrategy`, `minInstancesPerNode` для RF; `k`, `maxIter`, `tol`, `seed`, способ инициализации для KMeans.

2. Почему train, valid и test нельзя заменить одним случайным разбиением train/test во всех задачах? Во-первых, если по test подбирать гиперпараметры и порог, он перестаёт быть независимой оценкой — нужна отдельная valid. Во-вторых, случайное разбиение подходит только для независимых объектов: для временных рядов (Seoul Bike) модель должна учиться на прошлом и проверяться на будущем, а при нескольких строках на один объект (подкаст-эпизоды в самостоятельной части) нужно разбивать по группам. Иначе метрика завышена.

3. Что называется утечкой данных и почему она может дать очень высокую, но бесполезную метрику? Утечка — попадание в обучение сведений, которых не будет в момент прогноза, или информации из valid/test (признак, вычисленный из цели; статистики предобработки по всему набору; копии строк в разных частях; будущее время). Модель использует этот «подсказчик», метрика на отложенной выборке получается высокой, но в эксплуатации подсказки нет и качество резко падает.

4. Почему числовые коды Browser, Region и TrafficType рассматриваются как категории? Это номера-названия, а не величины: браузер 8 не «вдвое больше» браузера 4, между кодами нет порядка и расстояния. Если подать код как число, линейная модель найдёт ложную монотонную зависимость, поэтому применяются `StringIndexer` + `OneHotEncoder`.

5. Что обучают StringIndexer, StandardScaler и сам классификатор внутри Pipeline? `StringIndexer` — словарь «категория → индекс» по train (`handleInvalid="keep"` выделяет индекс для новых значений); `StandardScaler` — стандартные отклонения (и средние при `withMean=True`) координат по train; классификатор — свои параметры (коэффициенты LR или деревья RF). После `fit` всё это хранится в `PipelineModel` и одинаково применяется к valid, test и новым данным.

6. Почему accuracy около 0,85 почти ничего не доказывает для Online Shoppers? Покупок ~15,5 %, поэтому постоянный ответ «нет покупки» уже даёт accuracy ≈ 0,845 при recall = 0. Accuracy при дисбалансе определяется в основном частым классом; нужно смотреть precision, recall, F1 положительного класса и PR-AUC.

7. Как меняются precision и recall при повышении порога решения? Положительных решений становится меньше: recall не растёт (обычно падает — больше покупок пропускается), precision обычно растёт — остаются более уверенные решения, ложных срабатываний меньше (строгой монотонности precision нет). При понижении порога — наоборот.

8. Почему PR-AUC особенно полезна при редком положительном классе? PR-кривая строится по precision и recall, т.е. только по качеству нахождения редкого класса, и не «разбавляется»

большим числом TN, как FPR в ROC. Её базовый уровень равен доле положительного класса, поэтому PR-AUC честно показывает, насколько модель лучше случайного ранжирования.

9. Что показывают коэффициенты логистической регрессии и почему их нельзя считать причинными эффектами? Коэффициент — изменение логарифма шансов покупки при увеличении координаты на единицу (у нас — на одно стандартное отклонение) при фиксированных остальных. Это ассоциация в наблюдательных данных: признаки коррелированы (число страниц и длительность), есть неучтённые факторы, регуляризация сжимает и перераспределяет веса, one-hot-индикаторы зависят от выбора базовой категории. Причинность требует эксперимента или специальных методов.

10. Почему PageValues проверяется отдельным опытом? Как момент выдачи прогноза меняет список допустимых признаков? `PageValues` — самый сильный признак, и он может быть недоступен при раннем прогнозе (рассчитывается по накопленной сессии). Абляция показывает, насколько качество зависит от него. Допустимы только признаки, известные к моменту прогноза: для оценки по завершённой сессии поле допустимо, для прогноза в начале сессии — нет, и тогда ожидать прежнего качества нельзя.

11. Почему часы Functioning Day = No исключены из основной регрессии? В эти часы аренд 0, потому что сервис был закрыт, а не из-за погоды или времени. Это другой механизм; такие нули исказили бы зависимость спроса от признаков. Модель отвечает на вопрос «сколько аренд при работающем сервисе», а закрытие известно заранее и обрабатывается правилом.

12. Зачем час кодируется одновременно через sin и cos? Час цикличен: 23:00 и 00:00 соседние. На окружности (sin, cos) соседние часы оказываются рядом. Одна координата неоднозначна — одинаковое значение синуса бывает у двух разных часов (например, 6 − x и 6 + x), пара sin/cos задаёт час однозначно.

13. В каком смысле PolynomialExpansion делает зависимость нелинейной? Добавляются квадраты и попарные произведения исходных признаков. Модель остаётся линейной по коэффициентам (решается та же задача линейной регрессии), но прогноз становится нелинейной функцией исходных признаков — квадратичной, с взаимодействиями (например, «температура × час»).

14. Чем MAE отличается от RMSE? Какая метрика сильнее реагирует на пики ошибки? MAE — средний модуль ошибки, все ошибки весят линейно, метрика устойчива к выбросам. RMSE — корень из среднего квадрата ошибки, крупные промахи весят квадратично. Сильнее на пики реагирует RMSE; всегда RMSE ≥ MAE, и большой разрыв между ними говорит о наличии крупных ошибок.

15. Как понимать отрицательный R^2 на test? $R^2 = 1 - SSE/SST$, где SST считается вокруг среднего самой test-части. Отрицательное значение означает, что модель ошибается сильнее, чем постоянный прогноз, равный среднему факту test. Обычно это признак сдвига распределения (другой сезон и уровень спроса) или переобучения; так, «Среднее train» на Seoul Bike даёт $R^2 < 0$, потому что уровень спроса в valid/test отличается от train.

16. Почему простое среднее по часу является более сильным baseline, чем глобальное среднее? Оно уже воспроизводит суточный профиль спроса (ночной минимум, утренний и

вечерний пики), который объясняет большую часть дисперсии, и при этом использует только train. Сложная модель должна превосходить именно это содержательное правило.

17. Почему day_index может помочь на одном году и стать проблемой при переносе на другой? Внутри года `day_index` фиксирует общий тренд/сезон, и лес использует его для разбиений. Но деревья не экстраполируют: для дат за пределами train все значения попадают в последний интервал, и модель «замораживает» уровень конца обучающего периода. На другом году тренд может быть иным.

18. Почему trainingCost KMeans нельзя использовать как единственный критерий выбора k? Внутрикластерная сумма квадратов почти всегда убывает с ростом k (при k = n она равна 0), поэтому её минимум всегда даёт максимальное k. Нужны силуэт, «локоть», размеры кластеров и содержательная интерпретируемость профилей.

19. Что означает отрицательный, нулевой и положительный silhouette? s(i) = (b − a)/max(a, b). Положительный (ближе к 1) — объект ближе к своему кластеру, чем к соседнему, разделение выражено; около 0 — объект на границе двух кластеров; отрицательный — объект ближе к соседнему кластеру, назначение сомнительно.

20. Почему Revenue нельзя включать в features кластеризации, если после обучения мы хотим сравнить долю покупок? Тогда кластеры строились бы вокруг самой покупки, и различие долей покупок между кластерами было бы следствием построения, а не найденной структуры поведения — круговое рассуждение. Кластеризация превратилась бы в скрытую классификацию.

21. Почему PCA-проекция не доказывает, что кластеры полностью разделены в исходном восьмимерном пространстве? Две главные компоненты сохраняют только часть дисперсии; проекция может как наложить друг на друга кластеры, разделённые в других измерениях, так и визуально разделить перекрывающиеся. Это иллюстрация, а строгую оценку дают силуэт и профили в исходном пространстве.

22. Какие данные допустимо передавать через toPandas(), а какие должны оставаться распределёнными? В драйвер — только заранее ограниченные небольшие таблицы: метрики, агрегаты, кривые по valid/test в несколько тысяч строк, детерминированные выборки для графиков и силуэта. Исходные наборы, признаки, обучение и массовые предсказания остаются в Spark DataFrame.

23. Что необходимо сохранить вместе с PipelineModel, чтобы решение можно было применить повторно? Весь `PipelineModel` (преобразования + модель), а также паспорт применения: рабочий порог классификации, выбранные модели и k, seed, версию Spark, схему входных данных и правила подготовки вне Pipeline (фильтр Functioning Day, разбор даты, приведение типов, состав признаков и момент их доступности), а также метрики валидации для контроля дрейфа.

24. Какие отличия численных метрик между двумя версиями Spark допустимы, а какие указывают на ошибку? Допустимы небольшие расхождения (в третьем знаке, единицы процентов RMSE) из-за другой реализации оптимизатора, порядка суммирования, случайности деревьев и инициализации KMeans. На ошибку указывают: другое число строк в частях разбиения, другой состав

классов, резкое изменение метрик (десятки процентов), смена знака R^2, модель хуже baseline, «идеальные» метрики (признак утечки), другая размерность вектора признаков.

8. Общие выводы

1. **Классификация (Online Shoppers).** При дисбалансе 15,5 % accuracy неинформативна (0,845 у постоянного ответа). Случайный лес с весами классов превзошёл логистическую регрессию по PR-AUC (0,717 против 0,610 на valid). Порог 0,538 выбран по максимуму F1 только на valid; однократный test дал F1 0,654, recall 0,764, precision 0,571, PR-AUC 0,679. Абляция показала критическую зависимость от `PageValues` (PR-AUC 0,717 → 0,391): для раннего прогноза результат переносить нельзя.
2. **Регрессия (Seoul Bike).** Хронологическое разбиение и содержательные базовые правила обязательны: среднее по часу (RMSE test 440) сильнее линейной регрессии (544). Полиномиальное расширение снижает ошибку до 374, случайный лес — до 300 (R^2 0,715). Основные ошибки — занижение утренних и вечерних пиков; лес опирается на `day_index`, что ограничивает перенос на другой год.
3. **Кластеризация.** KMeans по 8 стандартизованным поведенческим координатам при k = 3 (силуэт 0,453) выделил «отказы» (0,4 % покупок), «обычные» (15 %) и «активные» сессии (23,9 %). Revenue использован только для описания после выбора k.
4. **Методика.** Все преобразования (словари категорий, импутация, масштабирование, полиномы) находятся внутри `Pipeline` и обучаются только на train; модели и пороги выбираются по valid, test используется один раз. Полные `PipelineModel` сохранены вместе с JSON-паспортом и после загрузки воспроизводят те же предсказания (расхождение score 0,0, кластеры совпадают).
5. **Самостоятельная работа (Podcast).** Найдены и устранены две точки утечки на уровне данных (полные дубли и повторяющиеся ключи); GBT (RMSE test 13,74 мин) вдвое лучше постоянного прогноза и на 3 % лучше сильного правила «доля × длительность». Главный резерв качества — восстановление пропущенной длительности эпизода.
6. **Особенности стенда.** Windows 11, local[*], 16 CPU, PySpark 3.5.3, Java 21. Метрики отличаются от контрольных значений методички (Spark 4.1) в пределах единиц процентов — это допустимые различия реализаций; состав частей разбиения совпал точно. Python-воркеры Spark на стенде аварийно завершаются, поэтому модели загружаются Java-ридером (на вычисления DataFrame/MLlib это не влияет: они выполняются в JVM).

```
for df in [pod_raw, pod_split, pod_train, pod_valid, pod_test, pod_test_pred]:
    df.unpersist()
spark.stop()
print("SparkSession остановлена.")
```

```
SparkSession остановлена.
```